第2章 数据的表示与运算

※主要内容

(1)数据的编码

机器数编码、十进制数编码(△)、字符编码，校验码(△)

(2)数据的表示

数据的表示方法，整数、实数、非数值数据的表示

(3)数据的运算

定点运算—逻辑运算，加减、关系、移位、乘法运算(方法/部件)

浮点运算—加减运算(方法)

(4)运算器的组成

运算器的功能&组成、部件组织、部件互连

※总体要求：

掌握数据的表示方法，可组织运算的方法及部件

§2.1 数据的编码

※主要内容：进制转换，机器数/十进制数/字符的编码，校验码

	十进制	二进制	八进制	十六进制
数 码	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	0, 1	0, 1, 2, 3, 4, 5, 6, 7	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
基数r	10	2	8	16
进 位	逢十进一	逢二进一	逢八进一	逢十六进一
	0	0	0	0
	1	1	1	1
	2	10	2	2
	3	11	3	3
	4	100	4	4
	5	101	5	5
	6	110	6	6
	7	111	7	7
	8	1000	10	8
	9	1001	11	9
	10	1010	12	A
	11	1011	13	B
	12	1100	14	C
	13	1101	15	D
	14	1110	16	E
	15	1111	17	F
	16	10000	20	10
	⋮	⋮	⋮	⋮

§2.1 数据的编码

※主要内容：进制转换，机器数/十进制数/字符的编码，校验码

	十进制	二进制	八进制	十六进制
数 码	0,1,2,3,4,5,6,7,8,9	0,1	0,1,2,3,4,5,6,7	0,1,2,3,4,5,6,7,8,9,*A*,*B*,*C*,*D*,*E*,*F*
基数*r*	10	2	8	16
进 位	逢十进一	逢二进一	逢八进一	逢十六进一
表 示	(数)$_{10}$	(数)$_2$	(数)$_8$	(数)$_{16}$
	$(101011)_{10}$	$(101011)_2$	$(101011)_8$	$(101011)_{16}$
	数*D*	数*B*	数*O*	数*H*
	101011*D*	101011*B*	101011*O*	101011*H*
C语言	数	0*b*数	0数	0*x*数

- B = Binary（二进制）O = Octal（八进制）
- D = Decimal（十进制）H = Hexadecimal（十六进制）

§2.1 数据的编码

※主要内容：进制转换，机器数/十进制数/字符的编码，校验码

	十进制	二进制	八进制	十六进制
数 码	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	0, 1	0, 1, 2, 3, 4, 5, 6, 7	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, *A*, *B*, *C*, *D*, *E*, *F*
基数*r*	10	2	8	16
进 位	逢十进一	逢二进一	逢八进一	逢十六进一
表 示	(数)$_{10}$	(数)$_2$	(数)$_8$	(数)$_{16}$
	$(101011)_{10}$	$(101011)_2$	$(101011)_8$	$(101011)_{16}$
	数*D*	数*B*	数*O*	数*H*
	101011*D*	101011*B*	101011*O*	101011*H*
C语言	数	0*b*数	0数	0*x*数

$$(1011.101)_2 = 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} = 11.625$$

$$(702.6)_8 = 7\times8^2 + 0\times8^1 + 2\times8^0 + 6\times8^{-1} = 450.75$$

$$(A9DE.CF)_{16} = A\times16^3 + 9\times16^2 + D\times16^1 + E\times16^0 + C\times16^{-1} + F\times16^{-2}$$

$$= 10\times16^3 + 9\times16^2 + 13\times16^1 + 14\times16^0 + 12\times16^{-1} + 15\times16^{-2} = 43486.80859375$$

§2.1 数据的编码

※主要内容：进制转换，机器数/十进制数/字符的编码，校验码

任意进制 ⟶ 十进制（按“权”展开）

【举例1】

$(1011.101)_2 = 1\times2^3+0\times2^2+1\times2^1+1\times2^0+1\times2^{-1}+0\times2^{-2}+1\times2^{-3} = 11.625$

$(702.6)_8 = 7\times8^2+0\times8^1+2\times8^0+6\times8^{-1} = 450.75$ $= 7\times(2^3)^2+0\times(2^3)^1+2\times(2^3)^0+6\times(2^3)^{-1}$

$(A9DE.CF)_{16} = 10\times16^3+9\times16^2+13\times16^1+14\times16^0+12\times16^{-1}+15\times16^{-2} = 43486.80859375$

$= 10\times(2^4)^3+9\times(2^4)^2+13\times(2^4)^1+14\times(2^4)^0+12\times(2^4)^{-1}+15\times(2^4)^{-2}$

常用2的幂次方数

2^{16}	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}
65536	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125

§2.1 数据的编码

※主要内容：进制转换，机器数/十进制数/字符的编码，校验码

1、进制及其转换 （又称数制）

- B = Binary（二进制）O = Octal（八进制）
- D = Decimal（十进制）H = Hexadecimal（十六进制）

*常用进制：二/八/十/十六进制，用后缀B、O、D、H表示

思考：十六进制中A~F的值如何记忆？C语言中16进制数如何表示？（前缀0x ）

*R进制→十进制数转换：按权相加 如$(10.1)_2=(1\times2^1+1\times2^{-1})_{10}=2.5D$

*十进制→R进制数转换：整数&小数分别转换 ←值的整数、小数均相同

除基取余/减权定位→┘ └←乘基取整 ←减权定位更快

例1—$10.25=(2^3+2^1)+(2^{-2})=1010.01B$

2 | 10	0	(低位)
2 | 5	1	↑
2 | 2	0	
2 | 1	1	(高位)
0	余数	

$10-2^4$ <0、
$10-2^3=2\geqslant0$，$k_3=1$
$2-2^2$ <0，$k_2=0$
$1-2^1=0\geqslant0$，$k_1=1$
$0-2^0$ <0，$k_0=0$

$0.25\times2=0.5$ 0(高位)
$0.5\times2=1.0$ 1(低位)

$0.25-2^{-1}$ <0，$k_{-1}=0$
$0.25-2^{-2}=0=0$，$k_{-2}=1$

*二-八-十六进制数转换：整数&小数分别转换，位数不够时两边补零

练习—$131.75_{10}=$ ＿＿＿＿ $_2=$ ＿＿ $_{16}$，D.EH= ＿＿＿＿ B= ＿＿ D

2、机器数及其编码

***数值数据：** （数学上的数）

表示格式—[符号] 数码 [小数点 数码]　　←[ ]表示可省略

如—$(-10.1)_2$、$(+10)_2$、$(-10)_2$、$(10)_2$

运算特征—符号与数值分开运算，减法先比较大小

方法一：符号与数值分开运算

场景：计算（-5）+（+3）

第一步：看符号

- 一个是负（-5），一个是正（+3）
- 符号不同 → 实际是做**减法**：5 - 3

第二步：比较绝对值大小

- |−5| = 5，|+3| = 3
- 5 > 3 → 所以结果取**较大者的符号**，即负号
- 数值部分：5 - 3 = 2

人的习惯

第三步：拼起来

- 结果 = **−2**

这就是“符号与数值分开运算，减法先比较大小”：

符号单独判断，数值单独算，减法前必须比大小决定谁减谁、结果是什么符号。

2、机器数及其编码

*数值数据：（数学上的数）

表示格式—[符号] 数码 [小数点 数码]　　←[ ]表示可省略

如—$(-10.1)_2$、$(+10)_2$、$(-10)_2$、$(10)_2$

运算特征—符号与数值分开运算，减法先比较大小

*机器数：指计算机内部用编码表示的数据

└——→┴→二进制，可表示符号、小数点及位置

如—$(+10)_2$→$(010)_2$、$(-10)_2$→$(110)_2$、$(-10.1)_2$→$(1010101)_2$

科学计数法$=-0.101\times2^2$

真值—指机器数表示的数学值

*机器数的编码：

目标—便于硬件实现运算　　如—符号与数值一起运算、减法不比较大小

类型—原码、补码、移码等，均为二进制编码

PPT约定：本章的数据默认是二进制、省略后缀B，
除非特殊说明 或 数码中含2~9

(1) 原码(sign-magnitude)表示法

*编码思想：机器数的最高位(符号位)用0/1表示+/−，
其余位(数值位)用真值的绝对值表示

*整数原码定义：$X=\pm x_{n-2}\cdots x_0$，x_i=0或1，$[X]_{原}=x_{n-1}x_{n-2}\cdots x_0$

例1—$[+110]_{原}=0110$，$[-110]_{原}=1110$；$[X]_{原}=1101$时X=−101

$[+000]_{原}=0000$，$[-000]_{原}=1000$　※$[+0]_{原}\neq[-0]_{原}$

*小数原码定义：$X=\pm 0.x_{-1}\cdots x_{-(n-1)}$，$[X]_{原}=x_0.x_{-1}\cdots x_{-(n-1)}$

└→恒为0→编码时省略→┘ └←硬件隐含表示

例2—$[+0.101]_{原}=0.101$；$[-0.101]_{原}=1.101$　(此处便于阅读)

若$[X]_{原}=1.01$，则X=−0.01

*原码的特性：

表示特性—$[X]_{原}$与X的表示范围相同，$[+0]_{原}\neq[-0]_{原}$

运算方法—符号与数值分开运算、减法先比较大小

└→不利于硬件实现←┴←使用减法运算

原码规则：最高位是符号位（0正1负），剩下的是数值本身。

- +5 = 0101（最高位0，数值101即5）
- -5 = 1101（最高位1，数值101即5）

优点：人类一看就懂，和十进制写法最像。

致命缺点：

1．0有两个表示：+0 = 0000，-0 = 1000（浪费一个编码）

2．加法要单独处理符号：不能直接加，比如 1 + (-1) = 0001 + 1001 = 1010（-2），答案是错的！

所以计算机一般**不用原码做运算**，只用来表示一些特殊场合（比如浮点数的小数部分）。

(2) 补码(two's complement)表示法

编码目标—①减法不比较大小 ②符号与数值一起运算

***模运算：** (适于定长运算)

定义—仅计量小于"模"的部分，丢弃其余部分

如—时针(模＝12)从10点拨到7点，10－3＝10＋9＝7(mod 12)

同余—若A＝B＋kM(k∈整数)，记A≡B(mod M)，称A和B为模M的同余

如—+3≡+19(mod 16)，-3≡+13(mod 16)

补数—若a＋b＝M，称a和b互为模M的补数

运算特征：减法可转化为加法 ∵c－a＝c－(M－b)＝c＋b(mod M)

***编码目标的实现策略：** (硬件为模运算)

绝对值的补数

目标①—正数用本身、负数用正补数表示 ←用加法实现减法

如—模8时，4＋(-6)≡4＋2 ←→ 0100＋1010

目标②—用0/1表示+/-，结果符号等于其值类型 ←补码运算时证明

如—模8时，4＋(-6)＝-2 ←→ 0100＋1010＝1110

4＋(-3)＝+1 ←→ 0100＋1101＝0001

*编码思想：机器数的最高位(符号位)用0/1表示+/-，
其余位(数值位)用本身或正补数表示

*整数补码定义：$X=\pm x_{n-2}\cdots x_0$，则模为2^n，$[X]_{补}=x'_{n-1}x'_{n-2}\cdots x'_0$

$$[X]_{补}=2^n+X \pmod{2^n}=\begin{cases}0\times 2^{n-1}+X & 0\leqslant X<2^{n-1}\\ 1\times 2^{n-1}+2^{n-1}-|X| & -2^{n-1}\leqslant X<0\end{cases}$$

例3—$[+001]_{补}=0001$，…，$[+111]_{补}=0111$

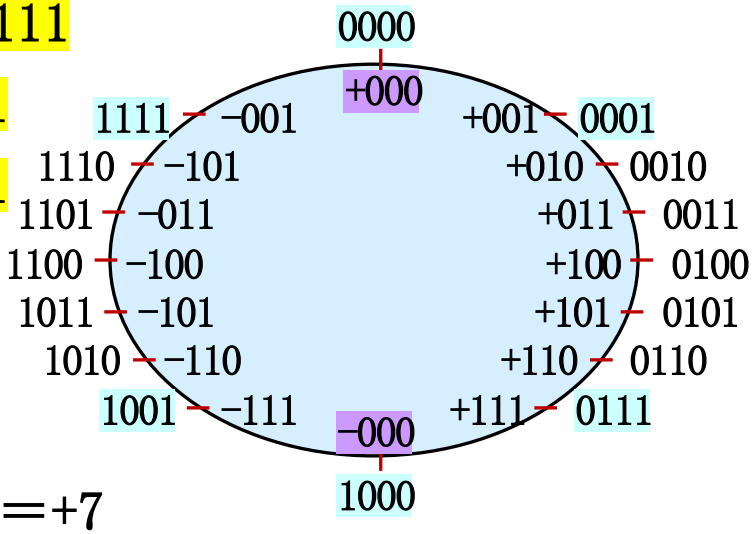
$[-001]_{补}=1000+(1000-001)=1111$

$[-111]_{补}=1000+(1000-111)=1001$

练习1—若A=-01100，求$[A]_{补}$

例4—$[+000]_{补}=0000$，$[-000]_{补}=0000$

※数0的补码唯一



例5—X≥0时，$[X]_{补}\leqslant 0111$，$X_{max}=2^{4-1}-1=+7$

X<0时，$[X]_{补}\geqslant 1000$，$X_{min}=-2^{4-1}=-8$

※补码比原码多表示1个数

*小数补码定义：$X=\pm 0.x_{-1}\cdots x_{-(n-1)}$，

则模为2，$[X]_{补}=x'_0.x'_{-1}\cdots x'_{-(n-1)}$

*补码的表示特性：

X与$[X]_{补}$的关系—设$X=\pm x_{n-2}\cdots x_0$，$[Y]_{补}=y_{n-1}y_{n-2}\cdots y_0$

① $X\geqslant 0$时，$[X]_{补}=0x_{n-2}\cdots x_0$

$y_{n-1}=0$时，$Y=+y_{n-2}\cdots y_0$

+1应不影响符号位
（不含$x_{n-1}\cdots x_0=10\cdots0$）

② $X<0$时，$[X]_{补}=2^n+X=[2^{n-1}+(2^{n-1}-1)+1]+X$

$=10\cdots0+(1\cdots1-x_{n-2}\cdots x_0)+1=1\overline{x_{n-2}}\cdots\overline{x_0}+1$

$y_{n-1}=1$时，$Y=[Y]_{补}-2^n=1y_{n-2}\cdots y_0-[2^{n-1}+(2^{n-1}-1)+1]$

$=2^{n-1}-2^{n-1}-[(1\cdots1-y_{n-2}\cdots y_0)+1]=-\overline{y_{n-2}}\cdots\overline{y_0}+1$

$X\rightarrow[X]_{补}$简便方法：

$X\geqslant 0$时，$[X]_{补}=0x_{n-2}\cdots x_0$

$-2^{n-1}<X<0$时，$[X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1$

$X=-2^{n-1}$时，$[X]_{补}=10\cdots0$

例6—若$X=+101$，$Y=-101$，则$[X]_{补}=0101$，$[Y]_{补}=1011$

$[Y]_{补}\rightarrow Y$简便方法：

$y_{n-1}=0$时，$Y=+y_{n-2}\cdots y_0$

$y_{n-1}=1$且$y_{n-2}\cdots y_0\neq0$时，$Y=-\overline{y_{n-2}}\cdots\overline{y_0}+1$

$y_{n-1}=1$且$y_{n-2}\cdots y_0=0$时，$Y=-2^{n-1}$

例7—若$[X]_{补}=0101$，$[Y]_{补}=1011$，则$X=+101$，$Y=-101$

*补码的表示特性：（续）

$[X]_{补}$与$[-X]_{补}$的关系—设$[X]_{补}=x_{n-1}x_{n-2}\cdots x_0$　←减法X－Y＝X＋(-Y)

X≥0时，$x_{n-1}=0$，$[X]_{补}=X$，

$$[-X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1=\overline{x_{n-1}}\,\overline{x_{n-2}}\cdots\overline{x_0}+1$$

$-2^{n-1}<X<0$时，$x_{n-1}=1$，$[X]_{补}=2^n-|X|$，

$$[-X]_{补}=-X=|X|=2^n-[X]_{补}=1\cdots1-x_{n-1}x_{n-2}\cdots x_0+1$$
$$=\overline{x_{n-1}}\,\overline{x_{n-2}}\cdots\overline{x_0}+1$$

$X=-2^{n-1}$时，$[X]_{补}=10\cdots0$，$[-X]_{补}$无法表示(需在运算时处理)

$(-2^{n-1}\leqslant X<+2^{n-1})$

$[X]_{补}\rightarrow[-X]_{补}$简便方法：

$[X]_{补}\neq10\cdots0$时，$[-X]_{补}=\overline{x_{n-1}}\,\overline{x_{n-2}}\cdots\overline{x_0}+1$

$[X]_{补}=10\cdots0$时，$[-X]_{补}$无法表示

例8—$[X]_{补}=0101$，$[Y]_{补}=1011$，则$[-X]_{补}=1011$，$[-Y]_{补}=0101$

补码规则：正数跟原码一样；负数 = 原码取反 + 1（符号位不变）。

关键性质：

1. **0只有一个表示**：0000（没有-0）

2. **减法变成加法**：1 + (-1) = 0001 + 1111 = 10000，4位溢出后剩下0000 = 0 ✅

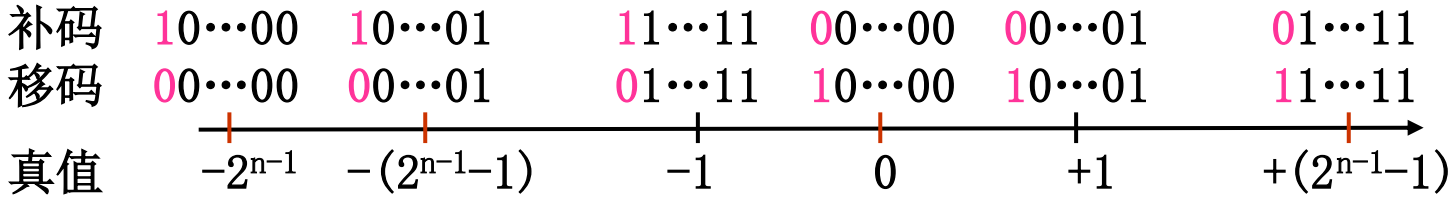
3. **加法器统一处理加减**：CPU里只需要一个加法器，不需要减法器

补码把“时钟”拨到负数那边。4位补码范围是 **-8 ~ +7**（不对称，负数多一个）。

(3) 移码表示法

编码目标—沿数轴方向，编码连续(机器数＝真值＋偏移量)

└←比较大小时不考虑符号



应用—表示浮点数的阶，故移码仅表示整数、仅含加/减运算

*移码定义：设$X=\pm x_{n-2}\cdots x_0$，则模$=2^n$、偏移量$=2^{n-1}$，

$$[X]_{移}=2^{n-1}+X \pmod{2^n}=2^{n-1}+X \qquad -2^{n-1}\leqslant X<+2^{n-1}$$

补码为2^n+X（负数的符号位＝1）

例9—$[-111]_{移}=0001$，…，$[-001]_{移}=0111$，$[\pm 000]_{移}=1000$，

$[+001]_{移}=1001$，…，$[+111]_{移}=1111$，$[-1000]_{移}=0000$

*移码的表示特性：①编码沿数轴连续(便于比较大小)

②容易与补码互换，$[X]_{移}=$符号位取反的$[X]_{补}$

练习3—

①若X＝+01001，$[X]_{原}$＝　　，$[X]_{补}$＝　　

　若Y＝-01010，$[Y]_{原}$＝　　，$[Y]_{补}$＝　　

②若$[X]_{原}$＝001010，X＝　　，$[X]_{补}$＝　　

　若$[Y]_{原}$＝101110，Y＝　　，$[Y]_{补}$＝　　

③若$[X]_{补}$＝001110，X＝　　，$[-X]_{补}$＝　　

　若$[Y]_{补}$＝101110，Y＝　　，$[-Y]_{补}$＝　　

④ X＝+01001时，$[X]_{原}$＝　　，$[X]_{补}$＝　　，$[X]_{移}$＝　　

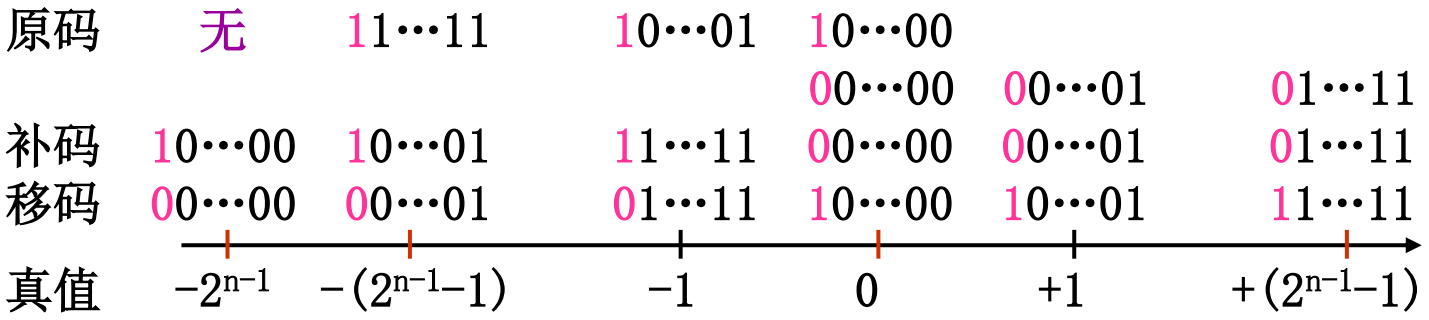
　Y＝-01010时，$[Y]_{原}$＝　　，$[Y]_{补}$＝　　，$[Y]_{移}$＝　　

原码、补码编码规则：（设$X=\pm x_{n-2}\cdots x_0$）

X≥0时，$[X]_{原}=0x_{n-2}\cdots x_0$；$[X]_{补}=0x_{n-2}\cdots x_0$，　$[-X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1$

X＜0时，$[X]_{原}=1x_{n-2}\cdots x_0$；$[X]_{补}=1\overline{x_{n-2}}\cdots\overline{x_0}+1$，$[-X]_{补}=0x_{n-2}\cdots x_0$

※机器数编码小结：



假设：$X=\pm d_{n-2}\cdots d_0$，

$[X]_{原}=x_{n-1}x_{n-2}\cdots x_0$，$[X]_{补}=y_{n-1}y_{n-2}\cdots y_0$，$[X]_{移}=z_{n-1}z_{n-2}\cdots z_0$

编码特征：

①x_{n-1}、y_{n-1}、z_{n-1}表示符号，$x_{n-2}\cdots x_0$、$y_{n-2}\cdots y_0$、$z_{n-2}\cdots z_0$表示数值

②$x_{n-1}=y_{n-1}=0/1$、$z_{n-1}=1/0$表示正/负

③$X\geqslant 0$时，$x_{n-2}\cdots x_0=y_{n-2}\cdots y_0=z_{n-2}\cdots z_0=d_{n-2}\cdots d_0$；

$X<0$时，$x_{n-2}\cdots x_0=d_{n-2}\cdots d_0$，$y_{n-2}\cdots y_0=z_{n-2}\cdots z_0=\overline{d_{n-2}}\cdots\overline{d_0}+1$

④补码/移码可表示-2^{n-1}，而原码无法表示

作业2-1：P84起3(1~4),4(1~2),5(1~3),6(1)

3、十进制数编码 （了解）

应用需求—I/O常为十进制，适于I/O密集型应用(如数据库)

编码目标—每个十进制数位直接用二进制编码表示

*编码方式：BCD码，即用4位二进制编码表示1位十进制数

└←Binary Coded Decimal，又称二-十进制编码

*BCD码的种类：分为有权码(8421/5211等)和无权码(格雷码/余3码等)，默认的编码种类为8421码

	0	1	2	3	4	5	6	7	8	9
8421码	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
余3码	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

*十进制数的表示：

格式— | 数值位 | … | 数值位 | 符号位 |，长度＝k×MEM单元长度

编码—数值位为BCD码，符号位为特殊编码(1100/1101/1111为+/-/无)

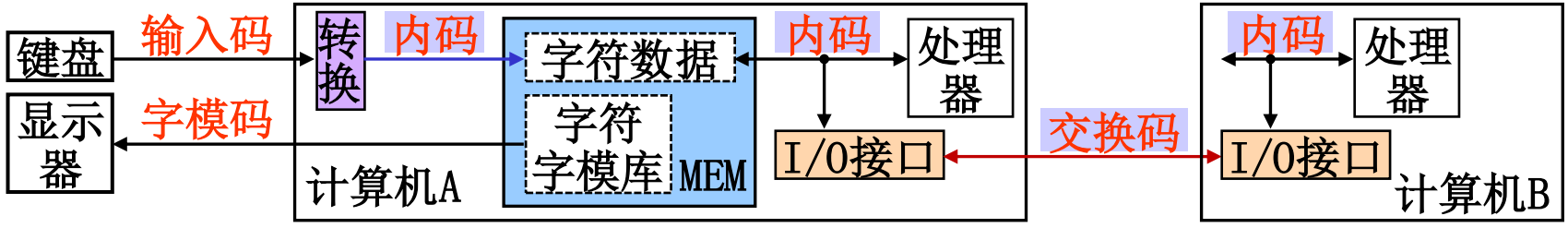
例— +427为0100 0010 0111 1100，-23为0000 0010 0011 1101，

426为0100 0010 0110 1111

4、字符编码 （了解）

指字符在字符集中的数字化代码(表示唯一的序号或特征号)

*字符编码的种类：有交换码、内码、输入码、字模码等



交换码—索引及传送时的编码(序号)，位数～字符集

内码—存储及处理时的编码(特征号)，位数～字符集&MEM单元长度

*常见的字符编码：（指交换码） ←内码常称为字符数据

编码类型	码点个数	编码长度	说明
ASCII码	128	7	美国标准信息交换码，英文，使用最广泛
EBCDIC码	256	8	扩展二-十进制交换码，英文，IBM定义
Unicode码	65536 2^{32}	16 32	统一字符码，有UCS-2、UCS-4两种字符集，支持各国语言，使用较广泛
ANSI码	256	8	美国国家标准协会交换码，英文，含ASCII码
GB2312-80	7445	14	汉字国标码，中文

场景：你在键盘上按一下 A 键，屏幕上显示 A

第 1 步：输入码（键盘按键 → 扫描码）

•键盘上每个键都有一个**物理位置编号**，叫“扫描码”（比如 A 键的扫描码是 0x1E）。

•你按下 A 键，键盘向计算机发送扫描码 0x1E。

•操作系统键盘驱动程序收到后，查表知道：哦，用户按的是 A 键。

✅ **输入码 = 扫描码 0x1E（键盘能识别的物理编号）**

第 2 步：输入码 → 内码（转换）

•操作系统根据当前输入法（英文），把 0x1E 映射成 ASCII 码。

•ASCII 表中，大写 A 的编号是 **65**（十进制），即十六进制 0x41。

•计算机把这个 0x41 存入内存，作为“当前用户输入了一个字符数据”。

✅ **内码 = 0x41（ASCII码），存储在内存中，供CPU处理**

场景：你在键盘上按一下 A 键，屏幕上显示 A

第 3 步：内码存储与处理（CPU 处理）

•内码 0x41 现在作为普通数据，参与程序逻辑。

•比如你在记事本里输入，程序会把多个字符的内码按顺序放到一个缓冲区（内存区域）里。

•此时 0x41 就是计算机内部**真正存储和处理**的形态。

✅ 内码 是计算机内部唯一使用的表示方式

第 4 步：内码 → 交换码（传输，可选）

•如果你要把这个文件**通过网络发送给朋友**，或者**保存到U盘**，计算机可能需要使用一种“交换码”格式。

•比如保存为 .txt 文件时，它依然保留 ASCII 码 0x41（因为 ASCII 本身就是交换标准之一）。

•如果发送到不同国家系统，可能需要转换成 Unicode 的交换格式（比如 UTF-8），以保证对方能正确识别。

✅ 交换码 = 在不同系统/设备间传输时使用的编码格式

第 5 步：内码 → 字模码（显示/打印）

•最终需要显示到屏幕时，操作系统拿着内码 0x41，去**字模库（字库）**里找对应的字形数据。

•字模库中，0x41 对应的是一组点阵数据，比如 8×8 像素的图案

•显卡读取这个字模数据，在屏幕上对应位置点亮/熄灭像素，你就看到了字母 **A**。

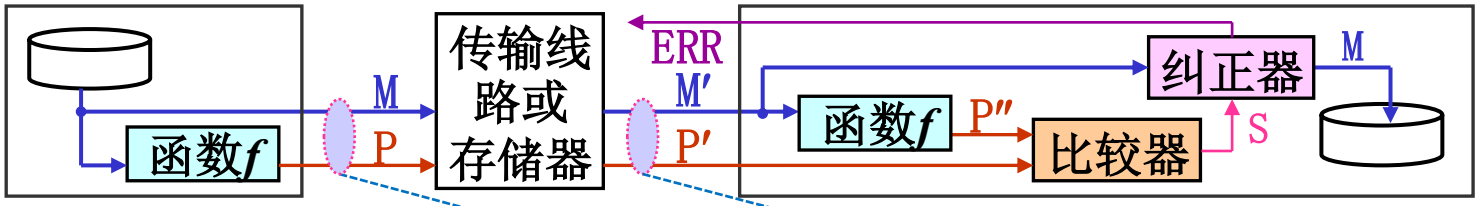
✅ 字模码 = 描述字符外形（如何画出来）的图形数据

5、校验码 （了解）

***数据校验过程：** --基于冗余校验思想

①用发送数据M 形成校验信息P，M与P一起传送

②用接收数据M′形成校验信息P″，比较P′及P″，校验数据(检/纠错)



结果—S＝0时正确(否则有错)，无法纠错时ERR＝1(否则＝0)

***校验码组成：** 数据位＋校验位，码距≥2(使合法码字数＜全部码字数)

码距—任意2个编码中位值不同的最小个数，如{000,011,101,110}的码距＝2

***校验原理：**

检错—判断接收码字∈合法码字？ ←发送码字为合法码字

纠错—基于非法码字定位错误→纠正错误(取反)

***检验码类型：** 奇偶校验码、汉明校验码，循环冗余校验码(CRC)

*奇偶校验码： 使校验码中1的个数为奇数/偶数个

校验码组成— 数据位$m_n\cdots m_1$ | 校验位p_1，校验位＝1位$(m_n\oplus\cdots\oplus m_1\oplus 1)$

校验能力— 可检测奇数个错误，无法纠错(不知道位置)

*汉明校验码： 数据分成多个有重叠的组，各个组采用奇偶校验编码

构成错误位置 ← 按校验码中位置加入 → 定位错误

校验能力— 自行约定，决定了校验位位数k、校验码组成

校验码组成—

位置	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
内容	m_{11}	m_{10}	m_9	m_8	m_7	m_6	m_5	p_4	m_4	m_3	m_2	p_3	m_1	p_2	p_1

单纠错码有$2^k-1\geqslant n+k$，能表示所有错误位置；

位置$h_k\cdots h_1$的数据位加入$h_i=1$的各个校验组

*循环冗余校验码： 使校验码能够被整除 ←除数G称为生成多项式

校验码组成— 数据位$m_n\cdots m_1$ | 校验位$r_k\cdots r_1$，k＝G的位数－1，

$r_k\cdots r_1=(m_n\cdots m_1\times 2^k)\div G$的余数 ←模2除(不关心商)

校验能力— 取决于G，检错较强，纠错很弱

奇偶校验码

例子（偶校验）：

要传数据 1010：

- 数一下：里面有 **2个1**（偶数）
- 如果采用**偶校验**，要求总1个数为偶数
- 现在已经是偶数了，所以校验位加 **0**
- 最终发送：1010 0

要传数据 1011：

- 里面有 **3个1**（奇数）
- 偶校验要求偶数，所以校验位加 **1**
- 最终发送：1011 1

接收方怎么查错？

- 收到 1010 0，数1的个数：2个 → 偶数 ✅ 正确
- 收到 1011 1，数1的个数：4个 → 偶数 ✅ 正确
- 如果传输中错了一位，变成 1010 1，数1的个数：3个 → 奇数 ❌ 发现错误

§2.2 数据的表示

※主要内容：数据表示方法，整数、实数、非数值数据的表示

*数据类型定义： 指一个值的集合，及定义在该值集上的一组操作

└→不同数据类型的值集或操作集不同←┘

如：C语言中的short（16位）与int（32位）

*应用中的数据类型：

- 数据
  - 数值数据
    - 无符号数 --自然数
    - 有符号数 --整数、纯小数、实数等
  - 非数值数据
    - 逻辑数 --布尔数
    - 字符(串) --含汉字
    - 其它 --声音、图形等

*硬件支持的数据类型： 最常用的数据类型(最基本的)， 由ISA约定

└→运算用硬件实现→┘←考虑性/价←┘

如：short、int、float、double，不支持数组、链表

*数据类型的转换： 由程序员、编译程序共同完成

应用中的数据类型 —程序员→ 编程语言中的数据类型 —编译程序→ 硬件中的数据类型

如：40位整数运算　　32位机的1个long int运算　　32位机的2个32位运算

编译程序会将一个 long int（64位）的运算，拆解成两次 32位 的运算（高32位算一次，低32位算一次）。

1、数据的表示方法

***数据的属性：数值—含进制/符号/小数点/编码(数值)/长度(可变)**

非数值—含编码(非数值)/长度(固定)

***硬件的特征：二进制，没有符号&小数点，长度固定(采用定长运算)**

矛盾的产生（数据的属性 vs 硬件的特征）为什么我们需要复杂的“数据表示方法”。

数据的属性（人类视角）：

- **数值数据**：很复杂。包含进制（十进制）、符号（+/-）、小数点、编码以及**长度可变**（比如数学里的整数可以无限大）。
- **非数值数据**：相对简单。比如字符、图像。包含特定的编码（如ASCII）和**固定长度**。

硬件的特征（机器视角）：

- 物理限制极其严格：只有**二进制**（高低电平）。
- 硬件根本不认识“正负号”，也看不见“小数点”。
- 硬件寄存器的位数是**固定**的（比如32位或64位，这叫**定长运算**）。

核心矛盾：如何把“长度可变、带符号、带小数点”的人类数据，塞进“只有二进制、无符号、无小数点、长度固定”的硬件盒子里？

*数据表示的要素：（面向存储及运算）

进制—用二进制表示 ←目前基本不支持二-十进制（BCD码），计算视角

符号—有符号数中用数字表示，无符号数中符号位置为数值

└——←长度应相等→——┘→利于存储&运算、扩大表示范围

小数点—用不同格式表示，格式有定点、浮点2种

└←约定点的位置 （整数、实数）

编码—有符号数为原码/补码/移码，无符号数为无符号编码

长度—为MEM单元长度的倍数(~存储)，每种数据有几种长度(~效率)

例如整数，有short，int

表示要素—进制＋格式＋编码，格式含长度，编码含符号

思考：非数值数据如何表示？ 同无符号整数(属性相近)

*数据表示的标识：　引子：机器数11011011的值是多少？

通过指令的操作码来指明(操作功能&数据类型，如整数加、浮点加)

*对数据运算的要求：　(每种数据类型的表示方法唯一[如长度固定])

运算方法基于表示方法组织(效率高&成本低)，

运算时进行溢出检测(保证正确性)　←软件处理溢出[灵活性好]

*数据表示与数据运算的关系：

数据的表示方法决定了数据的运算方法

└←计算机结构确定（ISA）　└←计算机组成实现

- 计算机结构（Computer Architecture）：是抽象的，它规定了“我们要支持补码整数”、“我们要支持 IEEE 754 浮点数”。这决定了**数据的表示**。

- 计算机组成（Computer Organization）：是具体的，它负责设计具体的加法器、乘法器、移位电路，来实现上面规定的运算。这实现了**数据的运算**。

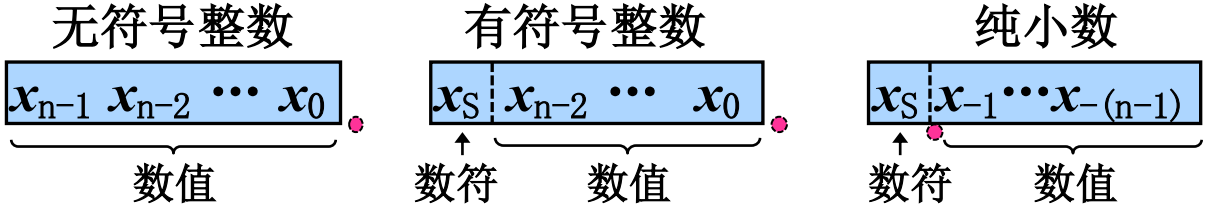
本课程的要求：认知数据的表示，实现数据的运算

2、整数的表示

(1)定点表示法

*基本思想：数据中小数点位置固定不变(导致可隐含表示)

*表示格式：



*表示范围：(假设格式长度为n位)

编码 \ 类型	自然数(无符号)	整数(有符号)	纯小数(有符号)
原码	—	$-(2^{n-1}-1)\sim+(2^{n-1}-1)$	$-(1-2^{-(n-1)})\sim+(1-2^{-(n-1)})$
补码		$-2^{n-1}\sim+(2^{n-1}-1)$	$-1\sim+(1-2^{-(n-1)})$
无符号编码	$0\sim2^n-1$	—	—

(2) 整数的表示

*表示方法：无符号整数—二进制、定点格式、无符号编码

有符号整数—二进制、定点格式、补码

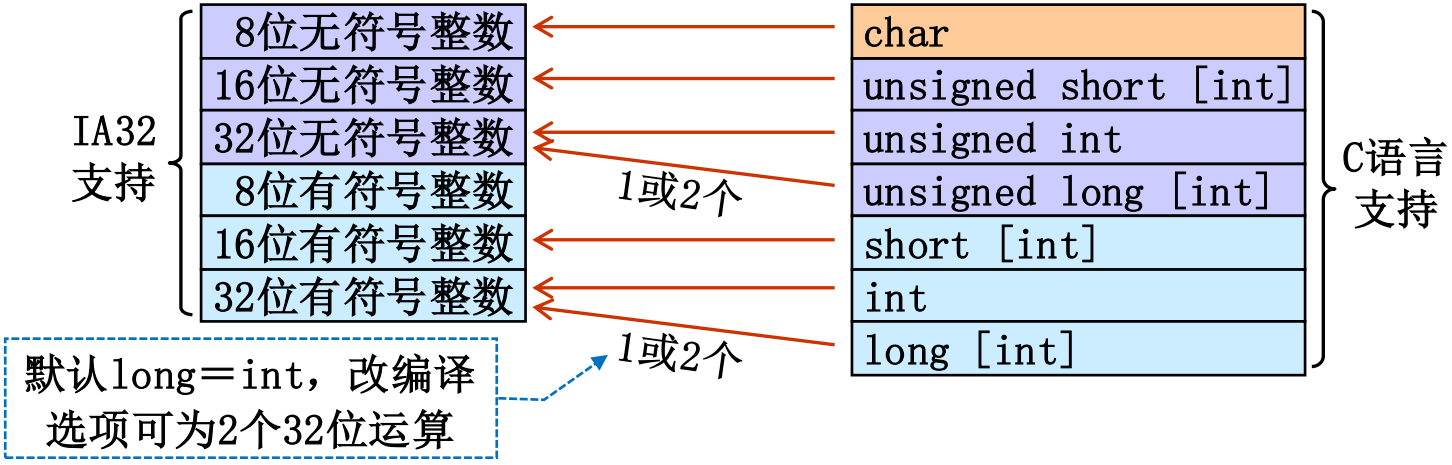
思考：为什么？

*硬件支持的整数类型：（取决于ISA约定）

均支持有/无符号整数，均有多种长度 ←长度＝k×MEM单元长度

└←基于使用频率、存储效率

例1—IA32 (Intel Architecture 32bit) 支持6种整数类型



例2—若int y=-1；unsigned short n=5；求IA32中y、n的机器数

y=FFFFFFFFH，n=0005H

(3)整数的类型转换

应用需求—语句及指令中，操作数的长度可能不同

如:C语言中，short si; int y,z=y+si; short sk=z;

如:指令R2←R2+imm中，R2为32位，imm为8位

转换者—编译器负责语句→指令、CPU负责指令→运算的转换

(如使用转换指令)　　(如使用转换操作)

*类型转换的方法:

长度相同时—解释为新类型

如:unsigned short usi=0xFFFE; short si=usi; //si=FFFEH

长度减少时—截断运算，仅保留低位(真值可能不变)

如:unsigned int uy=0x12345678; short si=uy; //si=5678H

长度增加时—位扩展运算，保持真值不变

如:unsigned short usi=2; short si=+2,sj=-2;

unsigned int ua=usi,uy=si; int z=sj;

//ua=00000002H，uy=00000002H，z=FFFFFFFEH

*位扩展运算的硬件实现：

运算功能—机器数(有/无符号整数)的位数增加、真值不变

运算方法— 零扩展时，机器数高位补0 ←面向无符号整数

符号扩展时，机器数高位补符号位 ←面向有符号整数

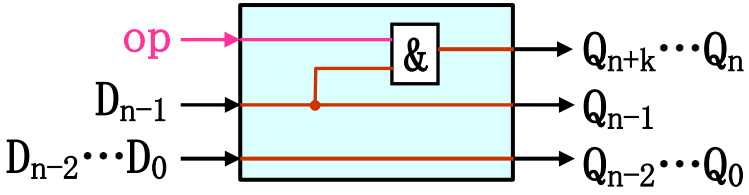
(补码表示)

	D=01101	D=11101
零扩展	Q=000 01101(13)	Q=000 11101(13)
符号扩展	Q=000 01101(+13)	Q=111 11101(-3)

逻辑实现—设入端$D_{n-1}\cdots D_0$、出端$Q_{n+k}\cdots Q_0$，控制端op(0/1=零/符号扩展)

输出函数：$Q_{n-1}\sim Q_0=D_{n-1}\sim D_0$，$Q_{n+k}\sim Q_n=op\cdot D_{n-1}$

内部逻辑：



思考：若电路仅含符号扩展功能，引脚&内部逻辑如何组织？

3、实数的表示

(1)浮点表示法

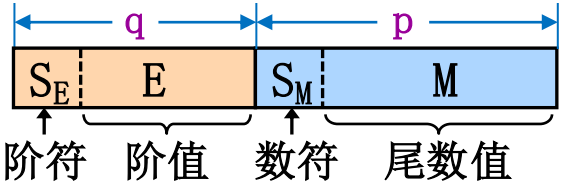
*基本思想：实数转换成$M\times R_M^E$，$M=\pm 0.m_{-1}\cdots m_{-p}$、，$E=\pm e_{q-1}\cdots e_0$，

尾数M为纯小数、基为R_M，阶E为整数、基为R_E；

小数点位置、R_M及R_E隐式表示、其余显式表示

└←目前均=2

*表示的一般格式：



，p+q=k×MEM单元长度

例1—假设浮点格式的尾数为11位、阶码为5位，尾数及阶均用补码表示，写出-54的浮点机器数

解：$-54=-110110B=-0.1101100000\times 2^{+0110}$，←位数不足时补0

浮点机器数为 00110 10010100000。

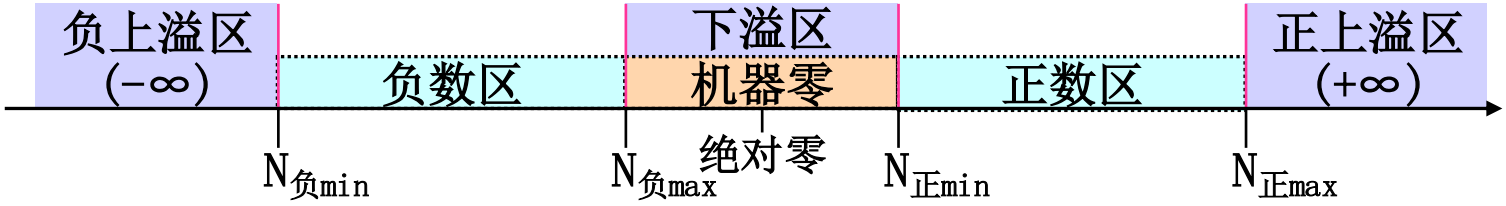
*表数范围及精度：（下页）

例2—浮点数的尾数为8位、阶为5位，填充下表空白处

阶	尾数	实数	浮点数的真值	阶码	尾数码
原码	原码	-0.0010111	$-0.1011100\times2^{-0010}$	1 0010	1 1011100
			$+0.1011010\times2^{-110}$	1 0110	0 1011010
移码	补码	-10101.11	$-0.1010111\times2^{+101}$	1 0101	1 0101001
			$-0.0011000\times2^{-1011}$	0 0101	1 1101000

练习—浮点数的尾数用7位补码、阶用5位移码表示，写出-54的浮点机器数，及浮点机器数645H的真值。

*表示范围与精度：（假设尾数、阶分别为p位、q位）



表示范围—$N_{正max}=-N_{负min}=(1-2^{-(p-1)})\times2^{+(2^{q-1}-1)}$，

$N_{正min}=-N_{负max}=2^{-(p-1)}\times2^{-(2^{q-1}-1)}$；范围受限于q

表示精度—相对误差(相邻尾数的一半)$=2^{-(p-1)}/2$；

精度受限于p，受具体表示影响(如例2最后一行少了2位)

(2)浮点数的规格化　—使浮点数的有效位数最大化

示例：尾数及阶均为4位，+111.0表示为$+0.111\times2^{+011}$优于$+0.011\times2^{+100}$

*规格化数：尾数真值的最高数值位为1，　←尾数范围常为-1.0＜M＜+1.0

即 0.5≤|M|＜1.0　←二进制为0.10…0～0.11…1

*规格化的操作：　(改变小数点的位置)

左规—尾数<<1位，阶－1　如：$-0.00101\times2^{+010}$左规2次为-0.1010×2^{000}

右规—尾数>>1位，阶＋1　如：$+1.0111\times2^{+010}$ 规右1次为$+0.1011\times2^{+011}$

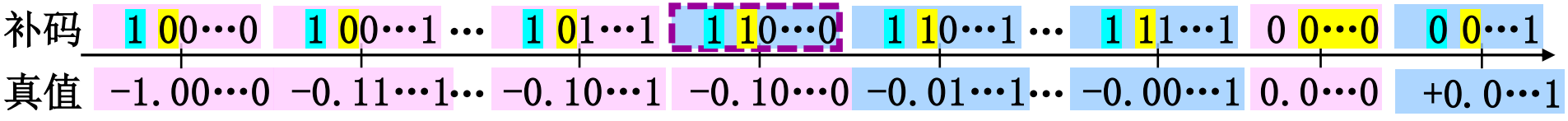
*规格化数尾数的数码特征：　(规格化操作的结束条件)

原码尾数—最高数值位为1　←原码表示范围为-1.0＜M＜+1.0

补码尾数—最高数值位与符号相反　←补码表示范围为-1.0≤M＜+1.0

├←正数（≥ 0.5)、＜-0.5的负数都是满足要求

┴←-0.5是特例（规格化会丢失精度）



(3) 实数的表示

*表示方法：二进制、浮点格式、尾数及阶单独编码(常不同)

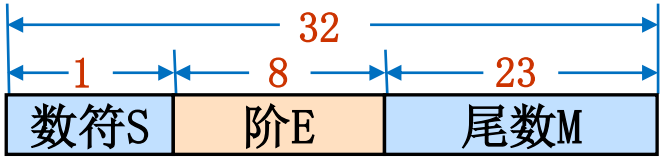
*硬件支持的浮点数类型：(取决于ISA约定)

均为IEEE 754标准，有单精度/双精度2种格式(含长度)

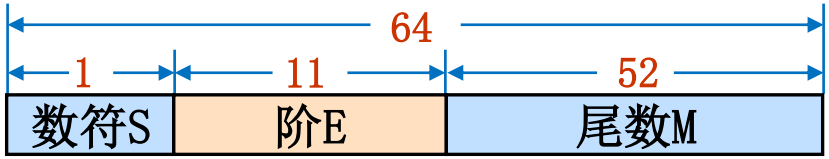
(4) IEEE 754标准

*进制：尾数M、阶E均为二进制 ←尾数精度最佳，阶(整数)无所谓

*表示格式：有单精度和双精度2种 ←数符与尾数分离，格式可扩充范围



单精度浮点格式



双精度浮点格式

*阶的编码：采用移码，偏移量＝标准移码－1 ←称余127码和余1023码

└→阶码E＝阶值＋($2^{8-1}-1$)＝阶值＋127

阶码约定—1≤E≤254表示是规格化数(即-126≤阶值≤+127)

(余127码例) E＝0及255表示是其它特征数(如非规格化数等)

*尾数的编码：(下页)

*尾数的编码：采用原码，规格化/非规格化数的小数点位置不同

非规格化尾数—真值＝±0.$m_{-2}\cdots m_{-24}$，编码＝S$m_{-2}\cdots m_{-24}$

└←±0.0*…*(精度≤22位)　└←0隐含表示，即±0.*…**(精度≤23位)

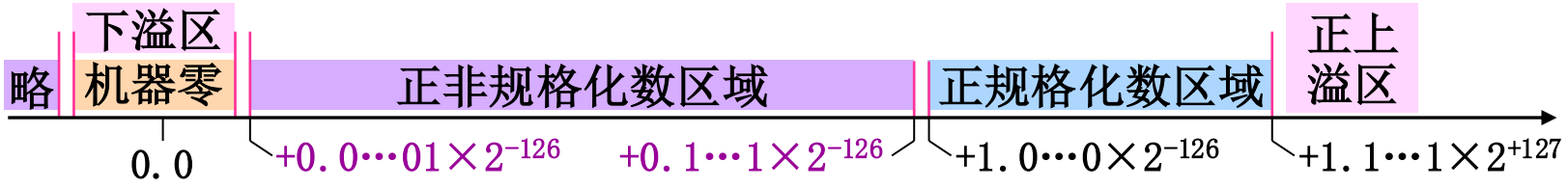
规格化尾数—真值＝±1.$m_{-2}\cdots m_{-24}$，编码＝S$m_{-2}\cdots m_{-24}$

└←±0.1*…*(精度＝23位)　└←1隐含表示，即±1.*…**(精度＝24位)

*表示特征：　(以单精度为例)

表示约定—

格式参数	真值N	说明
E＝0，M＝0	N＝±0	机器零(下溢区)
E＝0，M≠0	$N=(-1)^S\times2^{-126}\times0.M$	非规格化数
1≤E≤254	$N=(-1)^S\times2^{E-127}\times1.M$	规格化数
E＝255，M＝0	$N=(-1)^S\times\infty$	无穷大(上溢区)
E＝255，M≠0	N＝NaN	非数值



特征—可减少下溢区，可表示溢出、异常(如负数开根等)

例4—求IEEE 754单精度浮点数CC968000H的真值N

解：CC968000H＝1 10011001 00101101000000000000000，

N为规格化数(1≤E≤254)，

阶＝10011001B－(10000000-1)B＝+00011010B＝+26

尾数＝1.00101101B＝1.17578125

∴N＝(－1)^{1}×1.17578125×2^{+26}＝-1.17578125×2^{+26}

例5—求-11/128的IEEE 754单精度浮点数的机器码

解：-11/128＝(-1011)$_2$×2^{-7} ＝(-0.1011)$_2$×2^{-3}

[-4]$_{余127}$=[-4]$_{移}$－1

＝(-1.011)$_2$×2^{-4}＝(-1.011)$_2$×2$^{123-127}$

机器数为 1 01111011 01100000000000000000000

练习—写出-180的IEEE 754单精度浮点数的机器码

※数值数据的表示小结：　(指整数/实数)

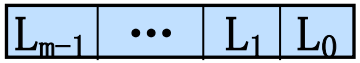
采用二进制，定点/浮点格式(几种长度)，某种/两种编码

4、非数值数据的表示

(1)逻辑数的表示

指逻辑数在存储/运算时的编码表示

*表示方法：二进制、向量格式、各分量为逻辑编码(1/0表示真/假)



*硬件支持的类型：有多种长度(与整数相同) ←长度＝k*MEM单元长度

└←逻辑运算复用算术运算部件

*硬件支持的运算：按位与、按位或、按位非等 ←对应C语言的&、|、~

(基于数据表示) 如—05H&03H=01H，05H|03H=07H

思考：C语言中&&、‖、!运算如何实现？

它们∉数据运算(硬件支持)，∈表达式运算(便于编程)，由编译器进行转换

如 a=(i>5 && j<6); 转换为
```
if (i<=5)       a=0;
else if (j>=6)  a=0;
     else       a=1;
```

(2) 字符的表示

指字符(交换码)在存储/处理时的编码表示，又称字符的内码

*表示方法： 二进制、定点格式、无符号编码 ←交换码为序号

*硬件支持的类型： 常支持几种字符，每种字符长度固定



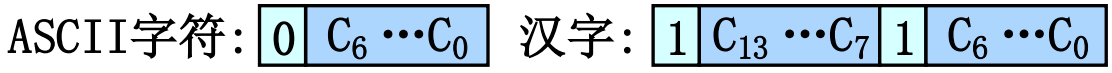
常见字符表示—

(设MEM单元＝1B)

字符集	交换码长度	内码长度	占用单元
ASCII	7	8位＝ 7+1	1个
GB2312-80	14	16位＝14+2	2个
Unicode	16或32	2B(UTF16)， 1B~6B(UTF-8)	1~6个

不同字符的识别— (均由软件处理)

ASCII字符与汉字： 常利用数据中的扩展码区分



Unicode与其他字符： 常利用程序中的字符长度区分

如：4B时为Unicode，1B/2B时为其他字符

*字符串的表示： 软件转换为字符操作(串结束用\0表示，串操作用循环实现)

1. ASCII（1963年）—— 只为英文设计

•**背景**：美国人发明计算机，只需要处理英文。

•**能力**：7位，128个字符，包括英文字母、数字、标点、控制符。

•**问题**：只能表示英文，**完全没有中文、日文、俄文等。**

2. GB2312（1980年）—— 中国自己打补丁

•**背景**：中国要用计算机，但ASCII没有汉字，怎么办？

•**办法**：自己搞一套汉字编码标准，同时**兼容ASCII。**

•**设计**：

- ASCII字符：1字节，最高位=0（0xxxxxxx）
- 汉字：2字节，每个字节最高位=1（1xxxxxxx 1xxxxxxx）

•**能力**：收录6763个常用汉字 + 682个符号。

•**问题**：只覆盖简体中文常用字，生僻字、繁体字、日文、韩文都没有。

3. Unicode（1991年）—— 全世界统一

•**背景**：每个国家都搞自己的编码（中国GB2312、日本JIS、韩国KS、欧洲ISO-8859），互相不兼容。同一串字节，在中国是“中”，在日本可能是别的字，**乱码严重。**

•**目标**：**给全世界所有字符，分配一个唯一的编号（码点）**，彻底解决冲突。

•**能力**：目前收录超过14万个字符，涵盖所有主要语言 + emoji。

•**实现方式**：

- **UTF-32**：每个字符固定4字节（简单但浪费空间）
- **UTF-16**：常用2字节，生僻4字节
- **UTF-8**：变长1~4字节，**兼容ASCII**（英文字母1字节，中文3字节）

(3) 媒体数据的表示　（不要求）

包括声音、图像、图形等复杂数据，运算有多种特殊类型

*数据的表示：仅支持基本数据类型的向量表示　←其余由软件转换

如：8×8像素块表示为8个向量，
每个向量含8个像素，每个像素为整数

*数据的运算：仅支持向量操作、部分特殊运算　←其余由软件转换

└←提高性能　└←如饱和运算

如：指令可处理一维向量，用循环(8次)实现像素块的运算，
饱和运算时的运算结果≥255时为255

※非数值数据的表示小结：（指字符/逻辑数/媒体数据）

二进制，定点/向量格式，某种编码

| └←逻辑数/媒体的分量占1位/多位，逻辑编码/基本数据编码

└————←字符，无符号编码

5、数据表示举例

*数据表示示例：

常见的数据表示方法—

数据 \ 表示		进制	格式(含长度①)	编码
数值数据	自然数	二	定点(几种长度)	无符号编码
	整数			补码
	实数		浮点(2种②长度)	尾数为原码②，阶为移码②
非数值数据	逻辑数		向量(长度同整数)	分量为逻辑编码
	字符		定点(长度固定)	无符号编码
	图像等		向量(长度固定/可变)	分量为无符号编码

注：①长度＝k×MEM单元长度；②基于IEEE 754标准；
③指每一种字符，多种字符时有多种长度

RISC-V的数据表示—8/16/32b整数(包含逻辑数&字符)，
32/64b浮点数，64/128b向量(早期)

IA32的数据表示—整数、浮点数、向量(同上)，指针、BCD数、位域

*数据表示与硬件参数的关联：（了解[融合知识点]）

数据类型长度—有多种(存储&运算效率高)，值为2^m位(可表示长度位数)

机器字长n—n＝max{各种整数类型的长度}

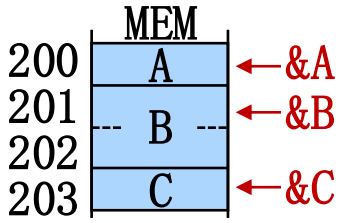
＝应该次数/实际次数

MEM单元长度w—关注存储效率&访问效率，侧重前者(后者可优化)

组织结果：w＝min{各种数据类型的长度}，均＝字节

数据存放：占用连续的MEM单元，

数据地址＝最小的MEM单元地址



大端方式—数据的最高有效字节存放在地址最小的单元，其余按序存放

小端方式—与大端方式方向相反，如：B_1B_0中B_0放在201#，B_1放在202#

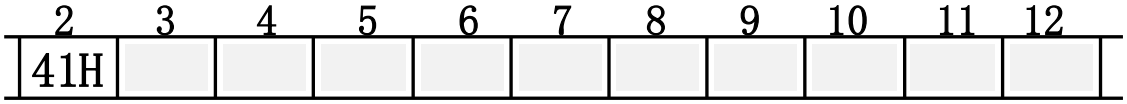
示例—IA32中，MEM按字节编址，数据采用小端方式存放。

右侧C语句中数据从2#单元起连续存放，存放结果？

```
char a='A';
int b=-36;
short c=b;
float e=40;
```

表示：机器数为 41H、　　、　　、

存放：



作业2-2：P85—14、18(1)

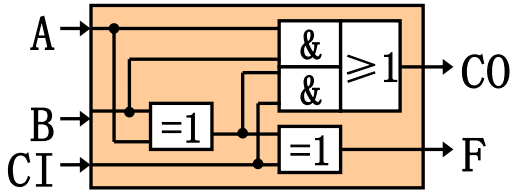
§2.3 定点数的运算

※主要内容：逻辑、加减&关系&移位&乘法的运算方法&部件组织
（针对有/无符号整数）

一、加法器与逻辑运算

1、加法器的组成

*1位全加器：（回顾）



A	B	CI	F	CO
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

真值表—（半加器没有CI）

内部逻辑— $F=(A\oplus B)\overline{CI}+(A\odot B)CI=A\oplus B\oplus CI$

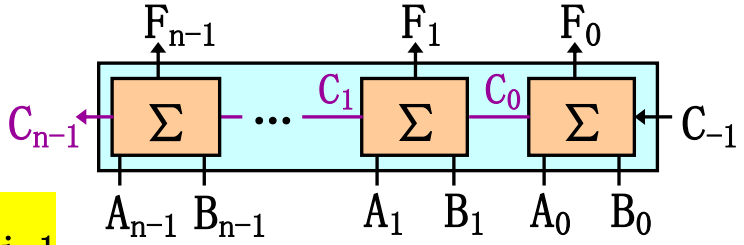
$CO=AB+(A\oplus B)CI$

*串行进位加法器：

组成—n个1位全加器＋串行进位电路

串行进位电路—串行形成$C_0\sim C_{n-1}$，

即$C_i=A_iB_i+(A_i\oplus B_i)C_{i-1}$



特点—进位电路简单，运算速度慢（$T_{加法}=n\times\Delta t$）

*并行进位加法器：

并行进位电路—同时形成$C_0 \sim C_{n-1}$ ←代入法，消除C_i与C_{i-1}的依赖关系

进位逻辑例：$C_0 = G_0 + P_0C_{-1}$ ←产生函数$G_i = A_iB_i$，传递函数$P_i = A_i \oplus B_i$

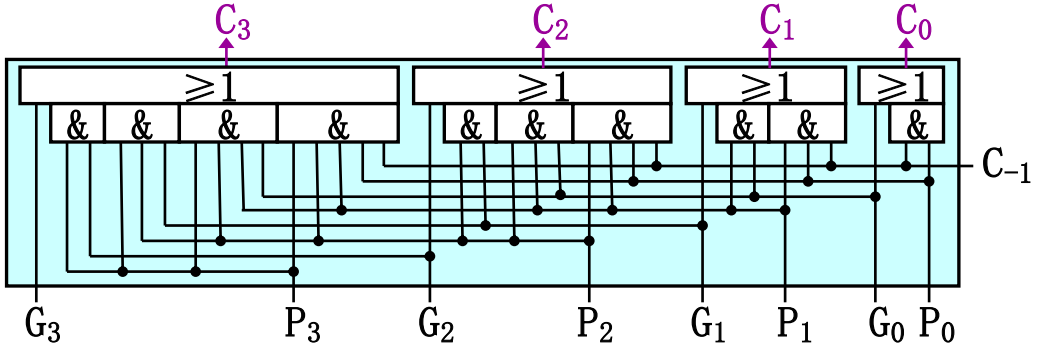
$$C_1 = G_1 + P_1C_0 = G_1 + P_1G_0 + P_1P_0C_{-1}$$

$$C_2 = G_2 + P_2C_1 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{-1}$$

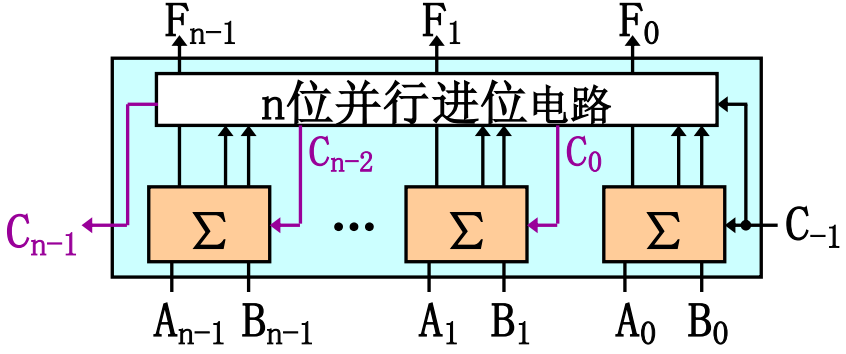
$$C_3 = G_3 + P_3C_2 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{-1}$$

进位电路例：

（成本急剧增加）

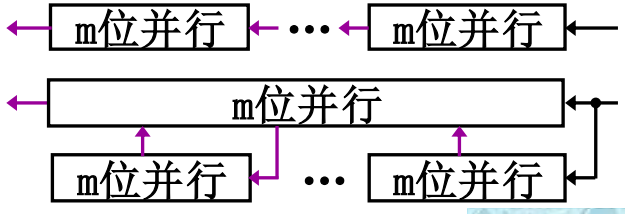


组成—n个1位全加器＋并行进位电路(有多种组织方式)



├←全并行，速度快&成本高

└←分组＋组间串行/并行



2、逻辑运算

*运算类型：与、或、非等

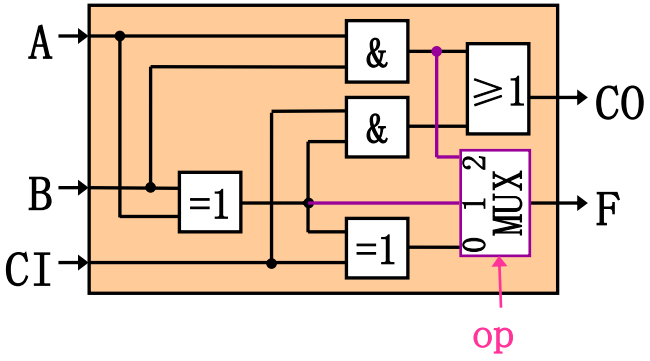
*运算方法：按位进行运算　　←基于逻辑数的向量表示格式

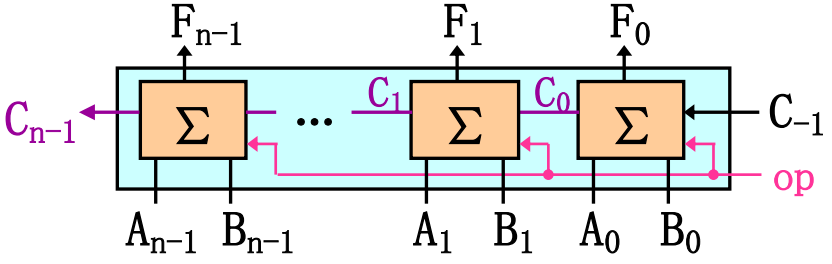
如：00000101&00000011=00000001，00000101|00000011=00000111

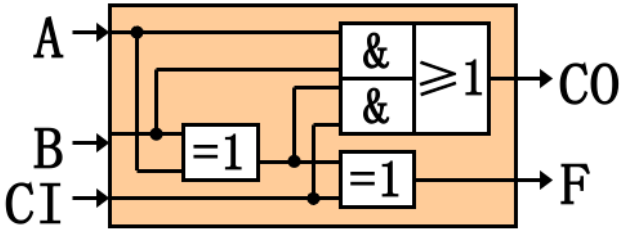
*逻辑实现：复用整数加法部件，在1位全加器出端增设选择器

　　└←加法用门电路实现

└←构成算术逻辑部件ALU







二、定点加减运算

运算类型—补码&无符号加减(整数)， 原码&移码加减(浮点数)

实现目标—尽量复用器件(降低成本)， 需要判断溢出(硬件为模运算)

1、补码加减运算

(1)运算方法 (结果为机器数)

*加法： $[A+B]_{补}=[A]_{补}+[B]_{补}$ ←推导：A+B+M=A+M+B+M (mod M)

*减法： $[A-B]_{补}=[A]_{补}+[-B]_{补}=[A]_{补}+\overline{[B]}+1$

*运算特征：符号与数值一起运算，减法不比较大小 ←基于公式可知

例1—设$[A]_{补}=11101$，$[B]_{补}=10111$，求$[A+B]_{补}$、$[A-B]_{补}$

解：$[A+B]_{补}=[A]_{补}+[B]_{补}=11101+10111=10100$

$[-B]_{补}=01001$，$[A-B]_{补}=[A]_{补}+[-B]_{补}=11101+01001=00110$

推广：同号相加&异号相减

丢弃进位(模运算)

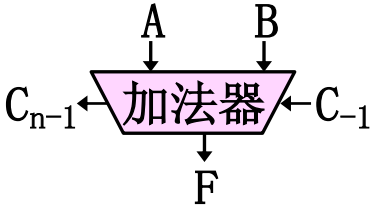
11101	$[-3]_{补}$	11101 $[-3]_{补}$
+ 10111	$[-9]_{补}$	+ 01001 $[+9]_{补}$
1 10100	$[-12]_{补}$	1 00110 $[+6]_{补}$

练习—整数用8位补码表示，A=+15，B=+24，求$[A+B]_{补}$、$[A-B]_{补}$

(2) 逻辑实现

运算分析—加法：$[A]_{补}+[B]_{补}=[A]_{补}+[B]_{补}+0$

减法：$[A]_{补}+[-B]_{补}=[A]_{补}+[B]_{补}+1$



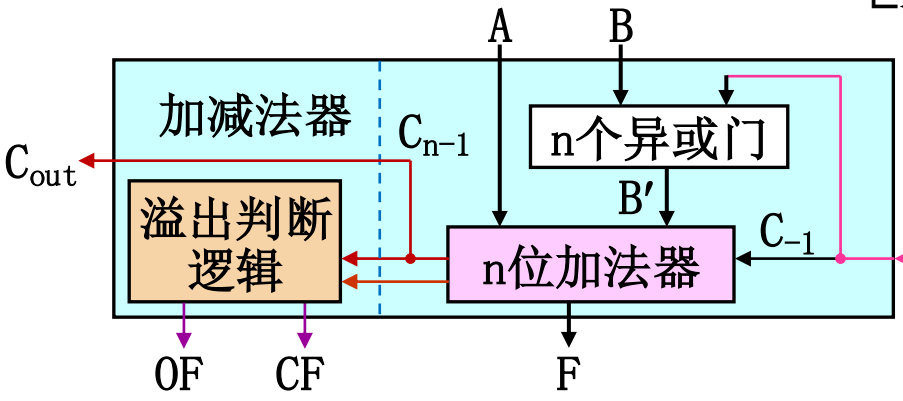
*实现思路：加/减法统一处理，用加法器运算，使$C_{-1}=0/1$(加/减时)

即 $[A\pm B]_{补}=[A]_{补}+B'+C_{-1}$，

其中，$C_{-1}=op$(0/1为加/减)，$B'=[B]_{补}\oplus C_{-1}$

*加减法器组成：加减逻辑+溢出判断逻辑(产生标志OF及CF)

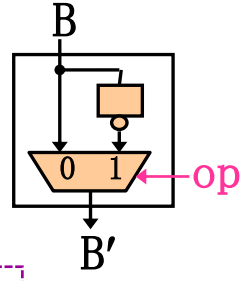
└←稍后讨论



OF—Overflow Flag
CF—Carry Flag

思考①：B′有其他实现方法吗？

非门+选择器，成本高



思考②：op=1表示ADD，加减法器有何变化？

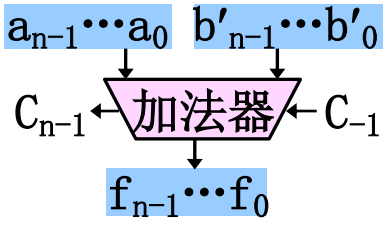
(3) 溢出判断　　—判断结果正确性

***溢出特征：** ①同号相加或异号相减时，结果与被加(减)数异号

②最高位进位及次高位进位中，进位只产生1个

5＋2或5－(-2)	-1＋(-4)或-1－(+4)	5＋4或5－(-4)	-5＋(-4)或-5－(+4)
0101 $[+5]_{补}$	1111 $[-1]_{补}$	0101 $[+5]_{补}$	1011 $[-5]_{补}$
+ 0010 $[+2]_{补}$	+ 1100 $[-4]_{补}$	+ 0100 $[+4]_{补}$	+ 1100 $[-4]_{补}$
0 0111 $[+7]_{补}$	1 1011 $[-5]_{补}$	0 1001 $[-7]_{补}$	1 0111 $[+7]_{补}$
↑ ↑	↑ ↑	↑ ↑	↑ ↑

***基于内部加法器判断：**　(判断与op无关)



方法①—使用符号位，$OF=\overline{a_{n-1}}\,\overline{b'_{n-1}}f_{n-1}+a_{n-1}b'_{n-1}\overline{f_{n-1}}$

$=(a_{n-1}\oplus f_{n-1})(a_{n-1}\odot b'_{n-1})$

方法②—使用进位位，$OF=C_{n-1}\oplus C_{n-2}$

例2—设$[A]_{补}=1010$，$[B]_{补}=0011$，判断$[A+B]_{补}$、$[A-B]_{补}$是否溢出。

解：$[A+B]_{补}=1010+0011=0\ 1101$，$OF=(1\oplus 1)(1\odot 0)=0$或$=0\oplus 0=0$

$[A-B]_{补}=1010+1101=1\ 0111$，$OF=(1\oplus 0)(1\odot 1)=1$或$=1\oplus 0=1$

练习—设A=+15，B=+24，用6位补码表示，判断A＋B、A－B是否溢出。

*基于加减法器的溢出判断：（判断与op有关）

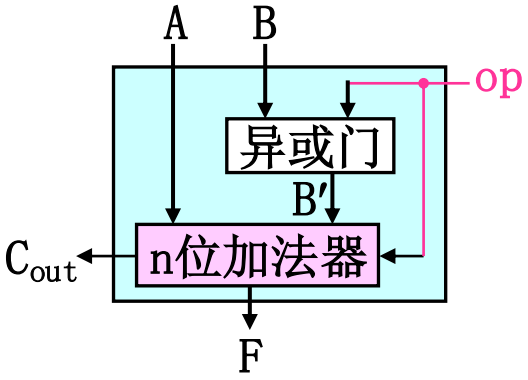
设 $A=a_{n-1}\cdots a_0$，$B=b_{n-1}\cdots b_0$，$F=f_{n-1}\cdots f_0$，

$op=0/1$（加/减）

则 $OF=\overline{op}(\overline{a_{n-1}}\,\overline{b_{n-1}}f_{n-1}+a_{n-1}b_{n-1}\overline{f_{n-1}})$

$+op(\overline{a_{n-1}}b_{n-1}f_{n-1}+a_{n-1}\overline{b_{n-1}}\,\overline{f_{n-1}})$

$=(a_{n-1}\oplus f_{n-1})(b_{n-1}\oplus op\odot a_{n-1})$ ←$b_{n-1}\oplus op$变减为加



特点—逻辑复杂（→OF通常在加减法器内部形成）

情况1：op = 0（加法）

$$OF=\overline{a_{n-1}}\,\overline{b'_{n-1}}f_{n-1}+a_{n-1}b'_{n-1}\overline{f_{n-1}}$$
$$=(a_{n-1}\oplus f_{n-1})(a_{n-1}\odot b'_{n-1})$$

- 第一项：A正、B正、结果负 → 正+正=负 → 溢出
- 第二项：A负、B负、结果正 → 负+负=正 → 溢出

情况2：op = 1（减法）

第一项：A正、B负、结果负 → 正-负=负 → 溢出

第二项：A负、B正、结果正 → 负-正=正 → 溢出

2、无符号加减运算

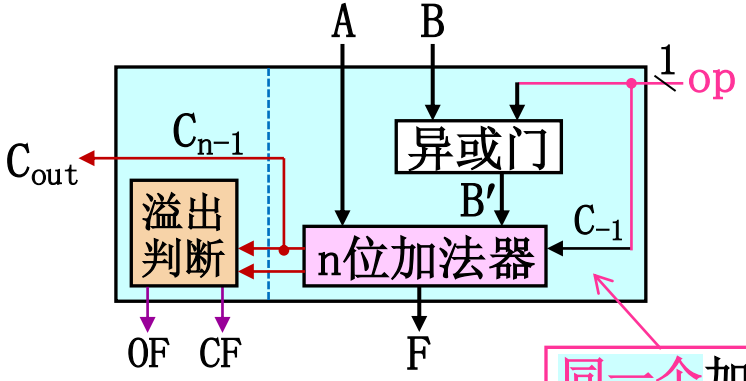
*运算方法：加法— $[A+B]_{无}=[A]_{无}+[B]_{无}=[A]_{无}+[B]_{无}+0$

减法— $[A-B]_{无}=[A]_{无}+[B]_{补数}=[A]_{无}+\overline{[B]_{无}}+1$

} 同补码加减

*逻辑实现：复用补码加减法器的运算逻辑，增加溢出判断逻辑

└←可降低成本　　└←不同于OF(稍后讨论)



思考①：支持无符号运算，op位数为何不变？

因为无符号和补码的运算逻辑完全相同，只是溢出判断不同，op只需要1位（0加1减），不需要区分有无符号。

思考②：无符号的溢出标志为何不复用OF？

有符号溢出（OF）：结果超出补码范围
无符号溢出（CF）：加法有进位，或减法有借位
如果复用OF，需要增加信号线来区分；不复用，直接增加CF，更简单。

同一个加减法器可实现有/无符号的加/减运算！

*溢出判断：　(CF称为进位/借位标记)

溢出特征—加法时有进位 或 减法时有借位

(op=0) (C_{out}=1)　(op=1) (C_{out}=0)

8+9	8−9	9−8
1000	1000	1001
+1001	+0111	+1000
1 0001	0 1111	1 0001

判断方法—外部为CF=$C_{out}\oplus op_{sub}$，内部为CF=$C_{n-1}\oplus C_{-1}$

练习—机器数A=001111，B=011000（无符号），求A+B、A−B的OF及CF

*综合例题：假定在一个8位字长的计算机中运行如下C程序段：

```
unsigned int x = 134, y = 246, z1 = x-y, z2 = x+y;
int m = x, n = y, k1 = m-n, k2 = m+n;
```

若编译时变量x、y、m、n、z1、z2、k1、k2分别分配在8个8位寄存器R1~R8中。请回答下列问题。

(1)执行上述程序段后，寄存器R1、R5和R6的内容分别是什么？

(2)执行上述程序段后，变量m和k1的值分别是什么？（十进制）

(3)上述程序段涉及带符号整数加/减、无符号整数加/减运算，这四种运算能否利用同一个加法器及辅助电路实现？简述理由。

(4)如何判断带符号整数加/减运算的结果是否发生溢出？上述程序段中，哪些带符号整数运算语句的执行结果会发生溢出？

(1)x＝86H，y＝F6H，z1＝0 90H，z2＝1 7CH，　←下画线为进位位

R1＝86H，R5＝90H，R6＝7CH

(2)m＝86H＝-122，n＝F6H，k1＝86H-F6H＝90H＝-112

(3)可以。减法可用加法实现，有/无符号加减的运算方法相同

(4)同号相加或异号相减，且结果与被加(减)数异号时溢出。

仅k2可能溢出(同号相加)，k2＝7CH，溢出

R1	x
R2	y
R3	m
R4	n
R5	z1
R6	z2
R7	k1
R8	k2

3、原码加减运算　（了解，不考）

运算特征—符号与数值分开计算，减法先比较大小

*运算方法：设$[A]_{原}=a_0.a_{-1}\cdots a_{-(n-1)}$，$[B]_{原}=b_0.b_{-1}\cdots b_{-(n-1)}$　←浮点运算

(1)判求和/求差—$op'=(a_0\oplus b_0)\overline{op}+(a_0\odot b_0)op=(a_0\oplus b_0)\oplus op$

(2)求和/求差—求差用加法实现，不够减时求补(先运算后比较大小)

op′	$[A\pm B]_{原}=f_0.f_{-1}\cdots f_{-(n-1)}$	进借位DF
0(求和)	$f_{n-1}=a_0$，$f_{-1}\cdots f_{-(n-1)}=a_{-1}\cdots a_{-(n-1)}+b_{-1}\cdots b_{-(n-1)}$	C_{-1}
1(求差)	$z_{-1}\cdots z_{-(n-1)}=a_{-1}\cdots a_{-(n-1)}+\overline{b_{-1}}\cdots\overline{b_{-(n-1)}}+1$， $C_{-1}=1$时(够减)，$f_0=a_0$，$f_{-1}\cdots f_{-(n-1)}=z_{-1}\cdots z_{-(n-1)}$ $C_{-1}=0$时(不够)，$f_0=\overline{a_0}$，$f_{-1}\cdots f_{-(n-1)}=\overline{z_{-1}}\cdots\overline{z_{-(n-1)}}+1$	0 1

*逻辑实现：加减法器＋辅助电路(判断/求补)

*溢出判断：$DF=(a_0\oplus b_0)\oplus op\oplus C_{-1}$　←溢出不是错，尾数可以左规/右规

4、移码加减运算　（不考，了解）

运算特征—类似于补码运算(符号取反)

*运算方法：　(浮点运算中讨论)

三、关系运算

运算类型—<、=等，用于有/无符号整数(如字符)　←常为转移条件

实现策略—逐位比较(比较器)、运算后比较(减法器＋逻辑门)，后者优

多种器件←┘　[相减＋判断] [已有]　└→多种逻辑

*运算方法：相减时产生结果标志，判断时基于结果标志　←不关心结果

结果标志—ZF(零)、CF(进位/借位)、SF(符号)、OF(溢出)，满足时＝1

如—4位机器数中，1111 (-1) －0100＝1 1011，ZF＝0，CF＝0，SF＝1，OF＝0

有符号补码　1011 (-5) －0100＝1 0111，ZF＝0，CF＝0，SF＝0，OF＝1

判断逻辑—

关系运算		A＝B	A＜B	A≤B	互补 A＞B	A≥B
①减法运算		A－B，产生标志(ZF/CF/OF/SF)				
②逻辑判断	无符号	ZF	CF	CF+ZF	$\overline{CF+ZF}$	$\overline{CF}$
	有符号	ZF	OF⊕SF	OF⊕SF+ZF	$\overline{OF⊕SF+ZF}$	$\overline{OF⊕SF}$

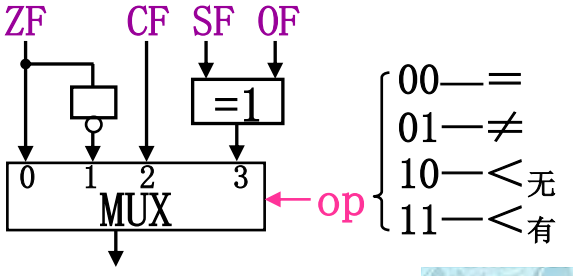
ZF—Zero Flag
SF—Sign Flag

*逻辑实现：

标志产生—$ZF=\overline{F_{n-1}+\cdots+F_0}$，$CF=C_{out}\oplus op_{SUB}$，

(基于相减)　$OF=C_{n-1}\oplus C_{n-2}$，$SF=F_{n-1}$

判断实现—组合逻辑电路(如右图)



三、关系运算

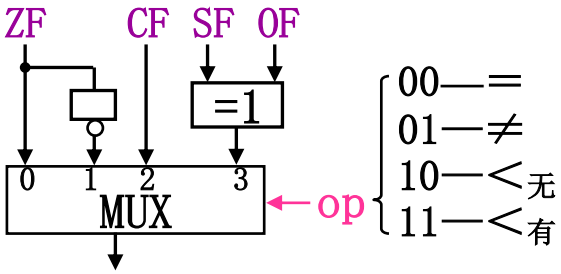
***逻辑实现：**

标志产生—（基于相减）

$ZF=\overline{F_{n-1}+\cdots+F_0}$，$CF=C_{out}\oplus op_{SUB}$，

$OF=C_{n-1}\oplus C_{n-2}$，$SF=F_{n-1}$

判断实现—组合逻辑电路(如右图)



- **标志产生逻辑**：
- $ZF=\overline{F_{n-1}+\cdots+F_0}$：所有结果位都为0时，ZF才为1。
- $CF=C_{out}\oplus op_{sub}$：加法看进位，减法看借位。
- $OF=C_{n-1}\oplus C_{n-2}$：最高位进位和次高位进位的异或，判断溢出。
- $SF=F_{n-1}$：结果的最高位。
- **判断实现电路（MUX 多路选择器）：**
- 根据输入的 op 指令（00=等于，01=不等于，10=无符号，11=有符号），MUX 会选择对应的标志位组合（比如CF，或者OF⊕SF），输出最终的比较结果。

四、移位运算

移位功能—A×2^k或A÷2^k，A为数据(有/无符号整数)，

k为移位位数(=$\log_2$数据长度)

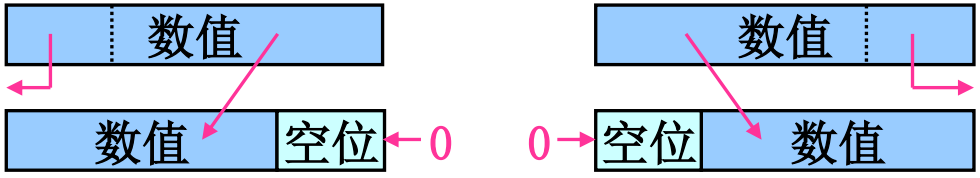
运算类型—逻辑左移、逻辑右移，算术左移、算术右移，等

符号表示：$<<_L$、$>>_L$，$<<_A$、$>>_A$ ←Logical shift及Arithmetic shift

1、逻辑移位运算 (面向无符号整数)

*运算方法：机器数整体移位；移出的数位丢弃，出现的空位补0

(无符号×2^k或÷2^k)



例1—设无符号整数的长度为8位，求57$<<_L$2、57$>>_L$2的机器数

解：$[57]_{无}$=00111001B，$[57<<_L2]_{无}$=11100100B=228

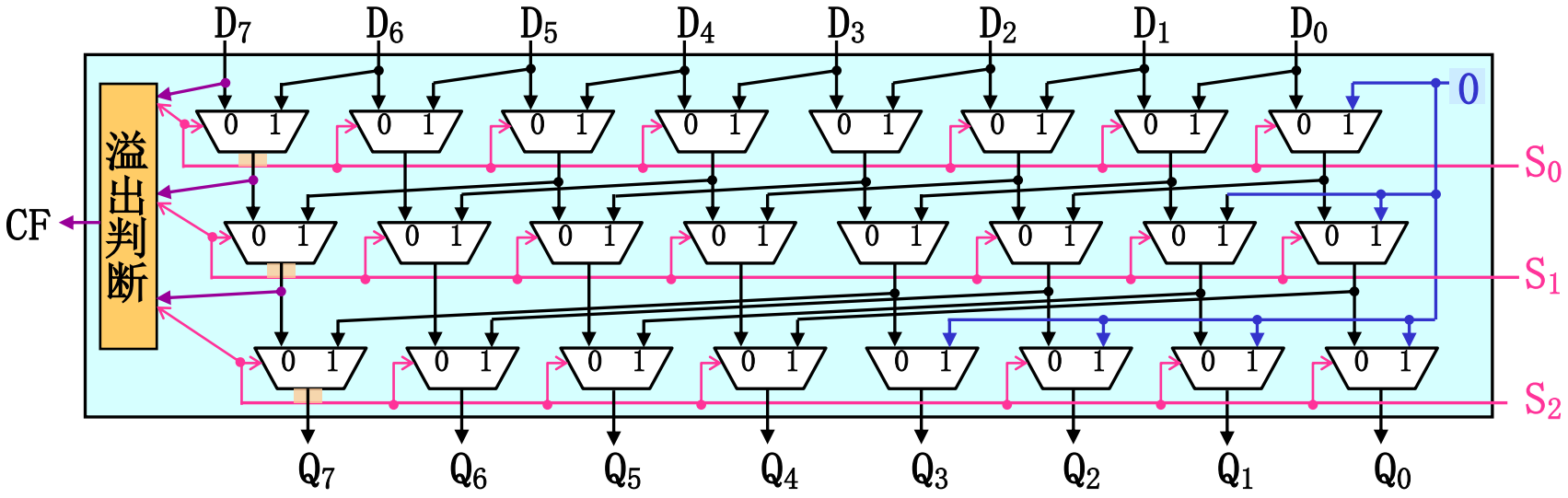
$[57>>_L2]_{无}$=00001110B=14

*逻辑实现：(下页)

*溢出判断：(下页)

*逻辑实现： 桶形移位器(组合逻辑，有左移/右移2种)

功能表：$Q_7 \sim Q_0 = D_7 \sim D_0 <<_L S_2S_1S_0$，如S＝001、010时，Q＝$D_6 \sim D_0 0$、$D_5 \sim D_0 00$



思考：为什么不使用移位寄存器？ 时延长(移动1位/CLK)、控制复杂

需要连上时钟CLK，每来一个时钟脉冲，只能移1位。如果要移7位，就要等7个时钟周期（时延长）。

*溢出判断：

溢出特征—设入端为$D_{n-1}\cdots D_0$、控制端为$S_{k-1}\cdots S_0$ ←位数$k=\log_2 n$

判断方法— 左移移丢码1(即真值1) ←右移移丢码1仅影响精度

$$CF=S_{k-1}D^*_{n-1}+\cdots+S_2D''_{n-1}+S_1D'_{n-1}+S_0D_{n-1}$$

$$=S_{k-1}(D_{n-1}+\cdots+D_{n/2})+\cdots+S_1(D_{n-1}+D_{n-2})+S_0D_{n-1}$$ ←倍增

如—n=8时，$CF=S_2(D_7+\cdots+D_4)+S_1(D_7+D_6)+S_0D_7$

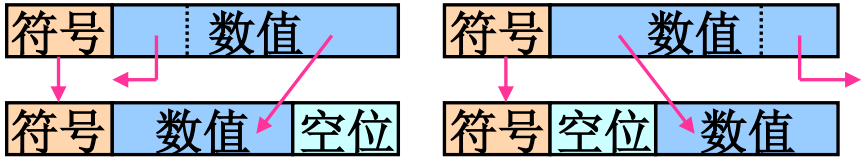
因为移位器是按1、2、4位排列的，所以只要某一位（比如移4位时）原本的最高位是1，它就一定会被移丢，产生进位。

- 代入n=8的例子：$CF=S_2(D_7+\cdots+D_4)+S_1(D_7+D_6)+S_0D_7$。
- 如果 $S_2=1$（移4位），只要 D_7,D_6,D_5,D_4 中有任何一个1，它就会被移出去，发生溢出。
- 如果 $S_0=1$（移1位），只要最高位 D_7 是1，就会溢出。

2、算术移位运算 （面向有符号整数）

*运算方法：机器数符号位不变、数值位整体移位；

移出的数位丢弃，出现的空位按编码特征添补



码制	符号位	左移的空位	右移的空位
原码	保持不变	补0	补0
补码		补0	补符号位

例2		原码&补码	原码	补码
		0011 (+3)	1011 (-3)	1101 (-3)
算术左移	先$<<_A1$	0110 (+6)	1110 (-6)	1010 (-6)
	再$<<_A1$	0100 (+4，溢出)	1100 (-4，溢出)	1100 (-4，溢出)
算术右移	先$>>_A1$	0001 (+1，损精度)	1001 (-1，损精度)	1110 (-2，损精度)
	再$>>_A1$	0000 (+0，损精度)	1000 (-0，损精度)	1111 (-1)

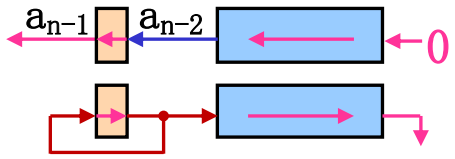
原码移位方法组织— 符号与数值分开计算，数值移位同逻辑移位

补码移位方法改进— 符号与数值位一起移位 ←可复用逻辑移位电路

（仅需考虑不溢出）

*逻辑实现： （下页）

*溢出判断： （下页）



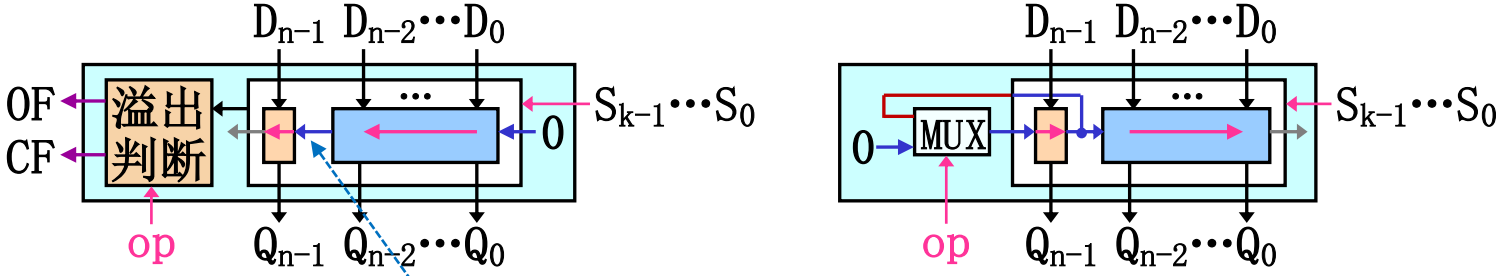
如 $0011<<_A1=0110$ （√）

$1101>>_A1=1110$ （√）

*逻辑实现：　(补码算术移位)

复用桶形移位器(逻辑移位)，增加溢出判断、右移移入值选择逻辑

└←不同于逻辑移位　└←0/符号位



移位器引脚组织—增加溢出信号CF及OF、控制信号op

└←原因同加减法器　└←选择移入值

*溢出判断：　(补码算术移位)

溢出特征—左移移丢与符号位相反的码(真值1)　←右移仅影响精度

判断方法—$OF=S_{k-1}(D^*_{n-2}\oplus D_{n-1})+\cdots+S_1(D'_{n-2}\oplus D_{n-1})+S_0(D_{n-2}\oplus D_{n-1})$

$=S_{k-1}(D_{n-2}\oplus D_{n-1}+\cdots+D_{n/2-1}\oplus D_{n-1})+\cdots$

$+S_1(D_{n-2}\oplus D_{n-1}+D_{n-3}\oplus D_{n-1})+S_0(D_{n-2}\oplus D_{n-1})$

如—n=8时，$OF=S_2(D_6\oplus D_7+\cdots+D_3\oplus D_7)+S_1(D_6\oplus D_7+D_5\oplus D_7)+S_0(D_6\oplus D_7)$

五、定点乘法运算

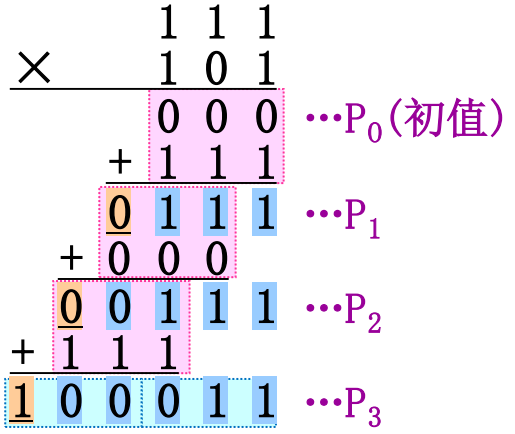
运算类型—无符号乘法、补码乘法(整数)， 原码乘法(纯小数)

实现目标—可基于加减法器(已有)实现 ←节约成本

*机器乘法的运算需求： (硬件为定长运算)

乘积由多次累加形成， ←只用一个加法器

部分积移位后再相加 ←数据迁就加法器



*机器乘法的实现思路：

①用循环的加法和右移运算实现

②加法时，加被乘数或0 ←实现1位乘

③右移时，部分积高位、部分积低位一起右移 ←低位不再参与运算

*机器乘法的类型： —基于每次相乘的位数

一位乘法、二位乘法、快速乘法

| | └←判断全部乘数，循环0次

| └←每次判断2位乘数，加0/A/2A/3A，右移2位；循环n/2次

└←每次判断1位乘数，加0/A，右移1位；循环n次

1、无符号乘法运算　　（用于整数，只讨论一位乘法）

$[A\times B]_{无}=[A]_{无}\times[B]_{无}$，A、B长度须相同，结果为2n位

***运算方法：**

$$A\times B=A\times b_{n-1}\times 2^{n-1}+\cdots+A\times b_1\times 2^1+A\times b_0\times 2^0$$

$$=2^n\times\{A\times b_{n-1}\times 2^{-1}+\cdots+A\times b_1\times 2^{-(n-1)}+A\times b_0\times 2^{-n}\}$$

$$=2^n\times\{(A\times b_{n-1}+(\cdots+(A\times b_1+(0+A\times b_0)\times 2^{-1})\times 2^{-1}\cdots)\times 2^{-1})\times 2^{-1}\}$$

部分积初值(递推所需)　　n个

递推公式— $P_1=(P_0+A\times b_0)\times 2^{-1}$　←初始部分积$P_0=0$

…

$P_i=(P_{i-1}+A\times b_{i-1})\times 2^{-1}$　←带进位右移(加法可能有进位)

…

$P_n=(P_{n-1}+A\times b_{n-1})\times 2^{-1}$　←循环n次

运算规则— 循环的判断-加法-移位操作，操作功能如下

判断	加法 （n位）	移位
$b_{i-1}=0$	$P'_{i高n位}=P_{i-1高n位}+0$，$P'_{i低位}=P_{i-1低位}$	$P_i=P'_i$带进位右移1位
$b_{i-1}=1$	$P'_{i高n位}=P_{i-1高n位}+A$，$P'_{i低位}=P_{i-1低位}$	

例1—设A=111、B=110，用无符号一位乘法求$[A\times B]_{无}$

解：需循环3次，过程如下

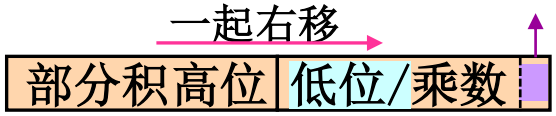
部分积高位	低位	乘数	说明
000		110	初始部分积$P_0=0$
+ 000 0 000 000	0	110	$B_0=0$，$P_{0高3位}+0$ P_1带进位右移1位
+ 111 0 111 011	10	110	$B_1=1$，$P_{1高3位}+A$ P_2带进位右移1位
+ 111 1 010 101	010		$B_2=1$，$P_{2高3位}+A$ P_3带进位右移1位

上页的运算规则参考：

判断	加法（n位）	移位
$b_{i-1}=0$	$P'_{i高n位}=P_{i-1高n位}+0$，$P'_{i低位}=P_{i-1低位}$	$P_i=P'_i$带进位右移1位
$b_{i-1}=1$	$P'_{i高n位}=P_{i-1高n位}+A$，$P'_{i低位}=P_{i-1低位}$	

*逻辑实现：

思路—①部分积低位放在乘数的空位上　←省1个REG



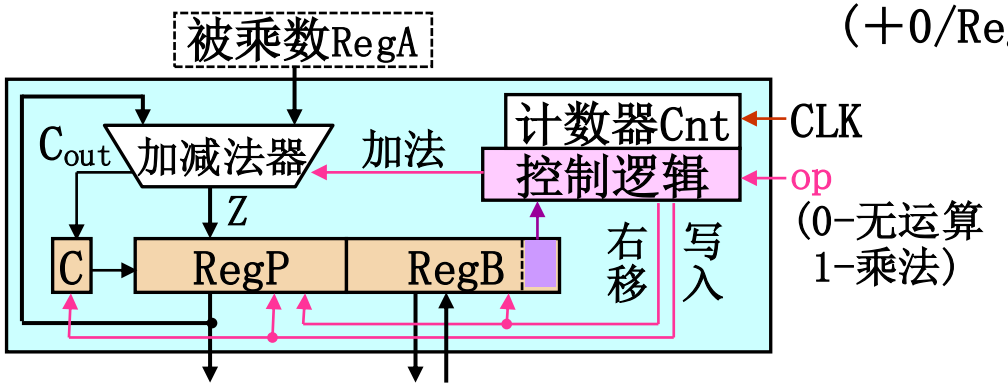
├→仅判断REG最低位　←实现简单

└→部分积与乘数一起右移　←附加要求

②循环的判断-加法-移位操作(判断b_0、＋0/A、带进位右移)

组成—判断逻辑(判断$RegB_0$)＋加法逻辑＋移位逻辑＋循环逻辑

(＋0/RegA)　(RegP及RegB)　(Cnt控制)



思考①：＋0如何实现？　支持Z＝A

思考②：1个CLK如何实现写&移位？

RegP仅写($C_{out}z_{n-1}\cdots z_1$)，C可省略

RegB仅移位(移入z_0)

控制逻辑—初始化(Cnt＝n、RegP＝0)，循环的判断-加法-移位操作

*溢出判断：　(满足指令中目的操作数为n位[如int=int*int]的需求)

判断逻辑—$OF=z_{2n-1}+\cdots+z_n$　←按n位结果判断，高位≠0时溢出

使用方式—硬件仅保存结果(不视为溢出)，软件需要时进行处理

1. 隐含的“双倍字长”结果（硬件其实没丢东西）

在真正的 CPU 硬件设计（如 x86、ARM 架构）中，乘法器**从来不会**只给你一个 n 位的结果。

- 当你执行 a * b（假设 a、b 都是 32 位）时，硬件会自动计算出一个 **64 位（2n位）** 的完整结果。
- 这个结果会分别存放在两个 32 位的寄存器中（比如 x86 中的 EDX 存放高 32 位，EAX 存放低 32 位）。
- **硬件层面：** 它**没有**做“截断”，它老老实实地把完整的 $2n$ 位结果都给你了。所以，硬件乘法器本身**永远不会溢出**。

2. 为什么 C 语言会“溢出”？（取决于你的代码怎么写）

溢出发生，是因为**你在软件层面主动要求它截断。**

- 如果你写 long long c = a * b;（假设 a、b 是 int），编译器会将低 32 位存入变量，**高 32 位会作为符号扩展自动填入** long long **的高位。**因为没有丢失信息，所以**不溢出**，结果完美。
- 如果你写 int c = a * b;（a、b 也是 int），结果也是 64 位的，但编译器**只把低 32 位塞进了变量 c 里，把高 32 位直接扔了！** 这时如果高 32 位里有 1，那 c 就错了，这就是**溢出。**

所以，你看到的“溢出”并不是硬件算错了，而是程序员主动告诉计算机：“我就只要一个 int 大小的结果，多的我不要了，你丢了它吧！”

2、补码乘法运算　　（用于整数，只讨论一位乘法）

$[A\times B]_{补}=[A]_{补}\times[B]_{补}$，A、B长度须相同，结果为2n位

*运算方法：又称Booth算法（布斯算法）

设$[B]_{补}=b_{n-1}b_{n-2}\cdots b_0$，则$B=b_{n-1}b_{n-2}\cdots b_0-2^n=-b_{n-1}\times 2^{n-1}+b_{n-2}\cdots b_0$

（正）$0+b_{n-2}\cdots b_0=+b_{n-2}\cdots b_0\rightarrow$ ⊥ $\leftarrow -2^{n-1}+b_{n-2}\cdots b_0=-(\overline{b_{n-2}}\cdots\overline{b_0}+1)$（负）

有$B=-b_{n-1}\times 2^{n-1}+b_{n-2}\times 2^{n-2}+\cdots+b_0\times 2^0$

$=-b_{n-1}2^{n-1}+(b_{n-2}2^{n-1}-b_{n-2}2^{n-2})+\cdots+(b_0 2^1-b_0 2^0)$　←基于$2^{i-1}=2^i-2^{i-1}$

$=(b_{n-2}-b_{n-1})2^{n-1}+(b_{n-3}-b_{n-2})2^{n-2}+\cdots+(0-b_0)2^0$　←0为初始比较值

基于补码特性，有$[A\times 2^{-1}]_{补}=[A]_{补}\times 2^{-1}$，

$[A\times B]_{补}=[A\times\{(b_{n-2}-b_{n-1})2^{n-1}+(b_{n-3}-b_{n-2})2^{n-2}+\cdots+(0-b_0)2^0\}]_{补}$

$=2^n\times[A\times\{(b_{n-2}-b_{n-1})2^{-1}+(b_{n-3}-b_{n-2})2^{-2}+\cdots+(0-b_0)2^{-n}\}]_{补}$

$=2^n\times[\{(b_{n-2}-b_{n-1})\times A+\{\cdots+$

$\{(b_0-b_1)\times A+\{0+(0-b_0)\times A\}\times 2^{-1}\}\times 2^{-1}\cdots\}\times 2^{-1}\}\times 2^{-1}]_{补}$

$=2^n\times\{[(b_{n-2}-b_{n-1})\times A]_{补}+\{\cdots+$

$\{[(b_0-b_1)\times A]_{补}+\{0+[(0-b_0)\times A]_{补}\}\times 2^{-1}\}\times 2^{-1}\cdots\}\times 2^{-1}\}\times 2^{-1}$

n次

递推公式—$[P_1]_{补}=\{[P_0]_{补}+[(b_{-1}-b_0)\times A]_{补}\}\times 2^{-1}$ ←附加位$b_{-1}=0$，$[P_0]_{补}=0$

…

$[P_i]_{补}=\{[P_{i-1}]_{补}+[(b_{i-2}-b_{i-1})\times A]_{补}\}\times 2^{-1}$ ←算术右移

…

$[P_n]_{补}=\{[P_{n-1}]_{补}+[(b_{n-2}-b_{n-1})\times A]_{补}\}\times 2^{-1}$ ←循环n次

运算规则—循环的判断-加法-移位操作，操作功能如下

判断	加法（n位）	移位
$b_{i-1}b_{i-2}=00$或11	$P'_{i高n位}=P_{i-1高n位}+0$，　$P'_{i低位}=P_{i-1低位}$	$P_i=P'_i$算术右移1位
$b_{i-1}b_{i-2}=01$	$P'_{i高n位}=P_{i-1高n位}+[A]_{补}$，　$P'_{i低位}=P_{i-1低位}$	
$b_{i-1}b_{i-2}=10$	$P'_{i高n位}=P_{i-1高n位}+[-A]_{补}$，　$P'_{i低位}=P_{i-1低位}$	

例2—$[A]_{补}$=011，$[B]_{补}$=101，用Booth算法求$[A\times B]_{补}$

思考：右移为何忽略加法进位？

循环过程中，+A、−A交替进行(2个10间必有01)，进位无用

如B=1 0 1 1 0

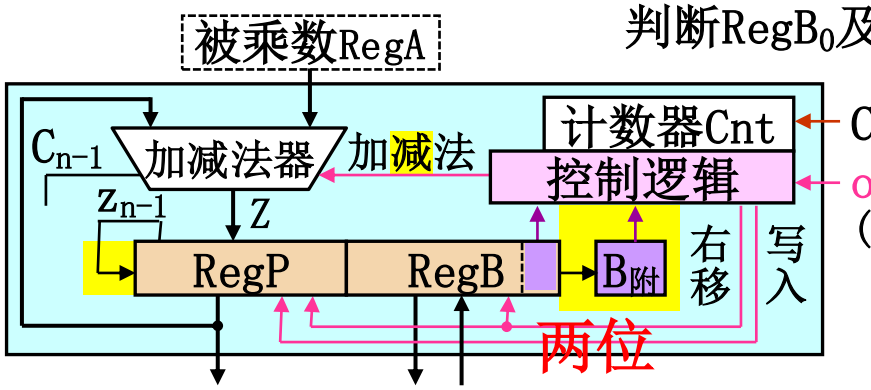
P高位	P低位	乘数	b_{-1}	说明
000		101	0	初始化P_0=0，b_{-1}=0
+ 101				b_0b_{-1}=10，+$[-A]_{补}$
101				
110	1	10	1	P_1算术右移1位
+ 011				b_1b_0=01，+$[A]_{补}$
001				
000	11	1	0	P_2算术右移1位
+ 101				b_2b_1=10，+$[-A]_{补}$
101				
110	111		1	P_3算术右移1位

*逻辑实现：

思路—①部分积低位放在乘数的空位上 ←原因同无符号乘法

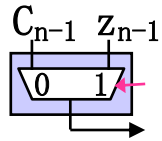
②循环的判断-加法-移位操作 ←类似于无符号乘法

组成—复用无符号乘法器，修改判断&移位逻辑



思考：如何集成为有/无符号乘法器？

电路—op改为2位，
移入值为C_{out}或z_{n-1}



控制—基于op判断&控制
(RegP仅写入、RegB仅移位)

控制流程—类似于无符号乘法器(判断/加法/移位有所不同)

*溢出判断：

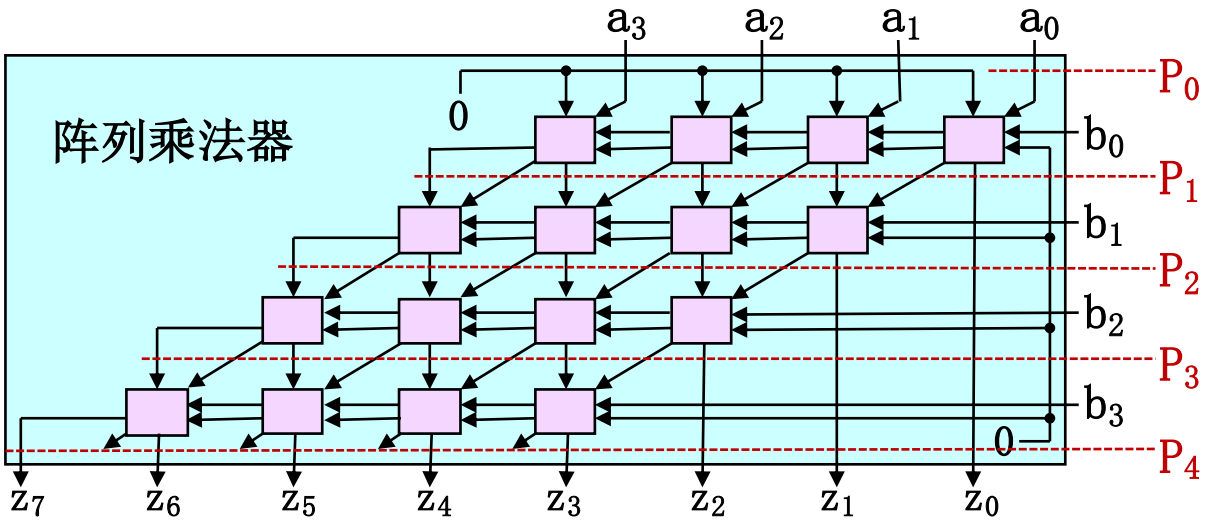
(满足指令中目的操作数为n位[如int=int*int]的需求)

判断逻辑—$OF=\overline{z_{n-1}}(z_{2n-1}+\cdots+z_n)+z_{n-1}(\overline{z_{2n-1}}+\cdots+\overline{z_n})$ ←高位≠符号位

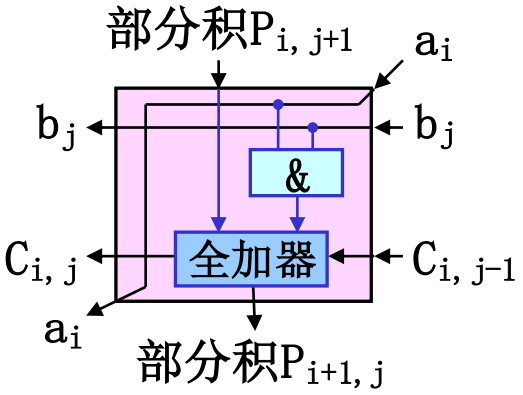
使用方式—硬件仅保存结果(不视为溢出)，软件需要时进行处理

3、阵列乘法器　　（快速乘法的实现）

***组成：** 组合逻辑电路(无需循环)，实现无符号乘法



位乘元件—$P_{i+1,j}=(a_i \times b_j + P_{i,j+1} + C_{i,j-1})$



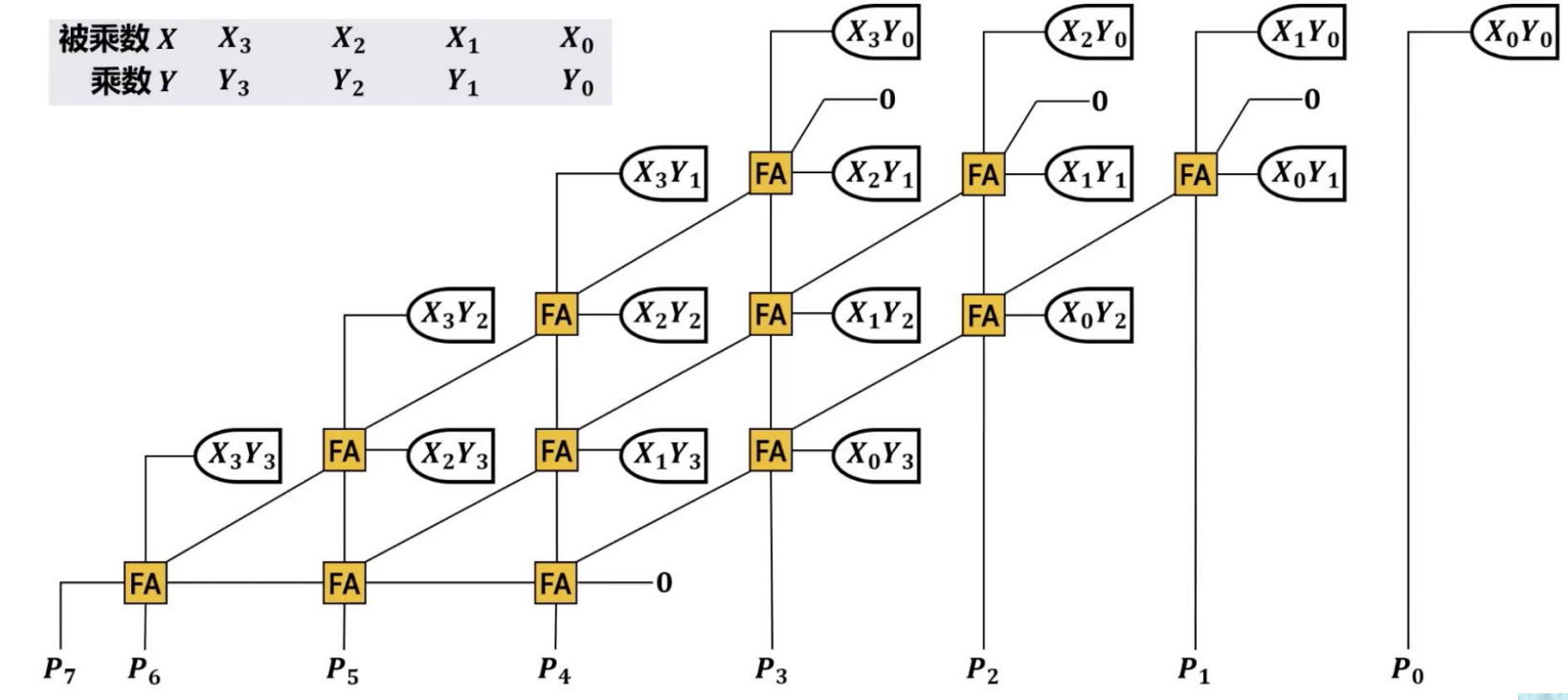
***性能：** $T_{阵列乘法} = n \cdot T_{位乘} < T_{一位乘法} = n \cdot T_{CLK}$

***应用：** 实现其他乘法需增加辅助电路

对比： 比“一位乘法”（移位+加法循环）快得多。因为一位乘法需要n个时钟周期，每个周期做一次加法和移位；而阵列乘法器的延迟主要这里是指它的关键路径延迟相当于来自门电路的传播延迟，通常只有几十纳秒。

作业2-3：P86—22(1/2)、24、25、27、30(1)、31(1)

被乘数 X	X_3	X_2	X_1	X_0
乘数 Y	Y_3	Y_2	Y_1	Y_0
×				
	X_3Y_0	X_2Y_0	X_1Y_0	X_0Y_0
X_3Y_1	X_2Y_1	X_1Y_1	X_0Y_1	
X_3Y_2	X_2Y_2	X_1Y_2	X_0Y_2	
X_3Y_3	X_2Y_3	X_1Y_3	X_0Y_3	
+				



※定点除法运算简介：　（了解，不考）

机器除法的要求—　（假设A÷B＝Q…R，B为n位）

```
      110
110 / 101001
      -110
       1000
       -110
        101
```

①被除数＝2n位(不够时自动补0)，商及余数＝n位，
余数符号与被除数相同　（小数除法不关心余数）

②运算前判断异常(又称操作数预判)，内容包括：

A＝0、B＝0时产生除法错异常，仅B＝0时产生除零异常，中止；

仅A＝0时商＝0，结束；

$|A_{高n位}|>|B|$时溢出[OF＝1]，中止。　←小数除法可左规/右规，继续

机器除法的实现思路—　（类似于机器乘法）

①用循环的左移和减法实现　←先左移后减法

②左移时，余数及被除数低位一起左移

③减法时，判断是否够减，据此上商及求余

机器除法的类型—　（基于中间除数的符号）

恢复余数法(不够减时加除数)、不恢复余数法(不够减时下轮减变加)

§2.4 浮点数的运算

※主要内容：浮点加减法的运算方法 （部件组织、乘除法均不要求）

*浮点数表示回顾：二进制，浮点格式，尾数&阶独立编码

IEEE 754标准：

S_M	E(移)	M(原)
1	8或11	23或52

一般格式：

E	S_M	M
k×MEM单元长度		

1、浮点加减运算

*运算步骤：假设$A=M_A\times 2^{E_A}$、$B=M_B\times 2^{E_B}$，$A\pm B=M_F\times 2^{E_F}$，$E_A\leqslant E_B$（小2大）

基本步骤—①对阶—$E_F=E_B$，$A=(M_A\times 2^{E_A-E_B})\times 2^{E_B}$

②尾数加减—$M_F=M_A\times 2^{E_A-E_B}\pm M_B$ （右移）←有效位数受损

③尾数规格化—使M_F为规格化数 ←表示精度最大化

运算精度的保证—运算过程中，增加尾数的位数(称为附加位)

如—IEEE 754标准的附加位至少有2位(保护位和舍入位)

实际步骤—包括对阶、尾数加减、尾数规格化、尾数舍入，

溢出判断包含在尾数规格化、尾数舍入步骤中

*运算方法：　（通过示例说明）

假设$A=M_A\times 2^{E_A}$、$B=M_B\times 2^{E_B}$，尾数加减后$F'=M'_F\times 2^{E'_F}$，

尾数规格化后$F''=M''_F\times 2^{E''_F}$，尾数舍入后$F=M_F\times 2^{E_F}$

例1—浮点数用一般格式表示，尾数、阶分别为5位补码、3位补码，附加位为2位，尾数舍入采用舍入法。

已知$A=0.1101\times 2^{+01}$、$B=-0.1010\times 2^{+11}$，求$[A+B]_{浮}$

①对阶—$\Delta E=E_A-E_B$，$\Delta E\geqslant 0$时$E'_F=E_A$，$M'_A=M_A$，$M'_B=M_B\times 2^{|\Delta E|}$，

否则$E'_F=E_B$，$M'_A=M_A\times 2^{|\Delta E|}$，$M'_B=M_B$

解-1：$[\Delta E]_{补}=001-011=110$，

$[E'_F]_{补}=011$，$[M'_A]_{补}=0.0011\ 01$，$[M'_B]_{补}=1.0110\ 00$

思考①：小数何时被大数吃掉(尾数右移后=0)？

②尾数加减—$M'_F=M'_A\pm M'_B$，尾数及附加位一起运算

解-2：$[M'_F]_{补}=0.0011\ 01+1.0110\ 00=1.1001\ 01$

思考②：运算结果溢出怎么办？　不算错误，尾数规格化可处理

③尾数规格化—$M''_F=M'_F$规格化结果，$E''_F=E'_F+k$；

判断E''_F是否溢出

规格化的操作：　（基于尾数的机器数特征）

$|M'_F|\geqslant 1.0$时—$M''_F=M'_F$右规1次，$k=1$　←因为$M'_F<2.0$

$0.5\leqslant|M'_F|<1.0$时—$M''_F=M'_F$，　$k=0$　←已为规格化数

$|M'_F|<0.5$时—$M''_F=M'_F$左规b次，$k=-b$　←直到$\geqslant 0.5$

溢出判断：　（基于阶的机器数特征）

判断阶码运算结果是否超出表示范围，溢出时进行处理

└←原码时需使用进位位，补码仅使用符号位

如—阶用8位补码表示时，阶码＞7FH或＜80H时溢出

IEEE 754标准—上溢时，$F=\pm\infty$(即$[E_F]_{移}=255,[M_F]_{原}=0$)

下溢时，$F=\pm 0$(即$[E_F]_{移}=0,[M_F]_{原}=0$)

解-3：因$[M'_F]_{补}=1.1001\ 01$，$[E'_F]_{补}=011$，

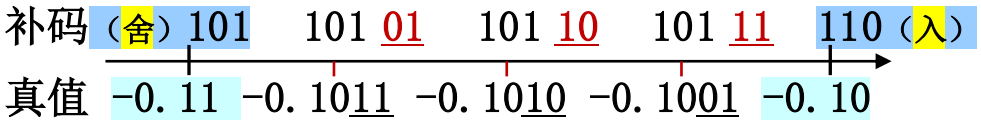
故左规1次，$[M''_F]_{补}=1.0010\ 10$，$[E''_F]_{补}=011-001=010$；

因$-4\leqslant[E''_F]_{补}\leqslant+3$，故$E''_F$未溢出

④尾数舍入——$M^*_F = M''_F + y$，y根据舍入方法得到；

$M_F = M^*_F$规格化结果($E''_F \rightarrow E_F$)，并判断E_F是否溢出

←同尾数规格化→



舍入方法：（基于真值）

	截断法	恒置法	舍入法
规则	朝0靠近(舍)	朝∞靠近(入)	朝最接近数靠近(舍/入)
结果	M_F不变	$M_F \pm 1$（正/负数）	$M_F \pm M_{F附加}$首位（正/负数）

IEEE 754标准—改进舍入法，改进：当$M''_{F附}$＝10…0时向偶数靠近

舍入处理：基于所选的舍入方法，对机器数进行处理

如—对补码正尾数，若$M''_{F附}$＝10、11，则$M^*_F = M''_F + 1$(入)；

对补码负尾数，若$M''_{F附}$＝00、01，则$M^*_F = M''_F$(入)

解-4：因$[M''_F]_{补} = 1.0010\ 10$，$[E''_F]_{补} = 010$，尾数采用舍入法，

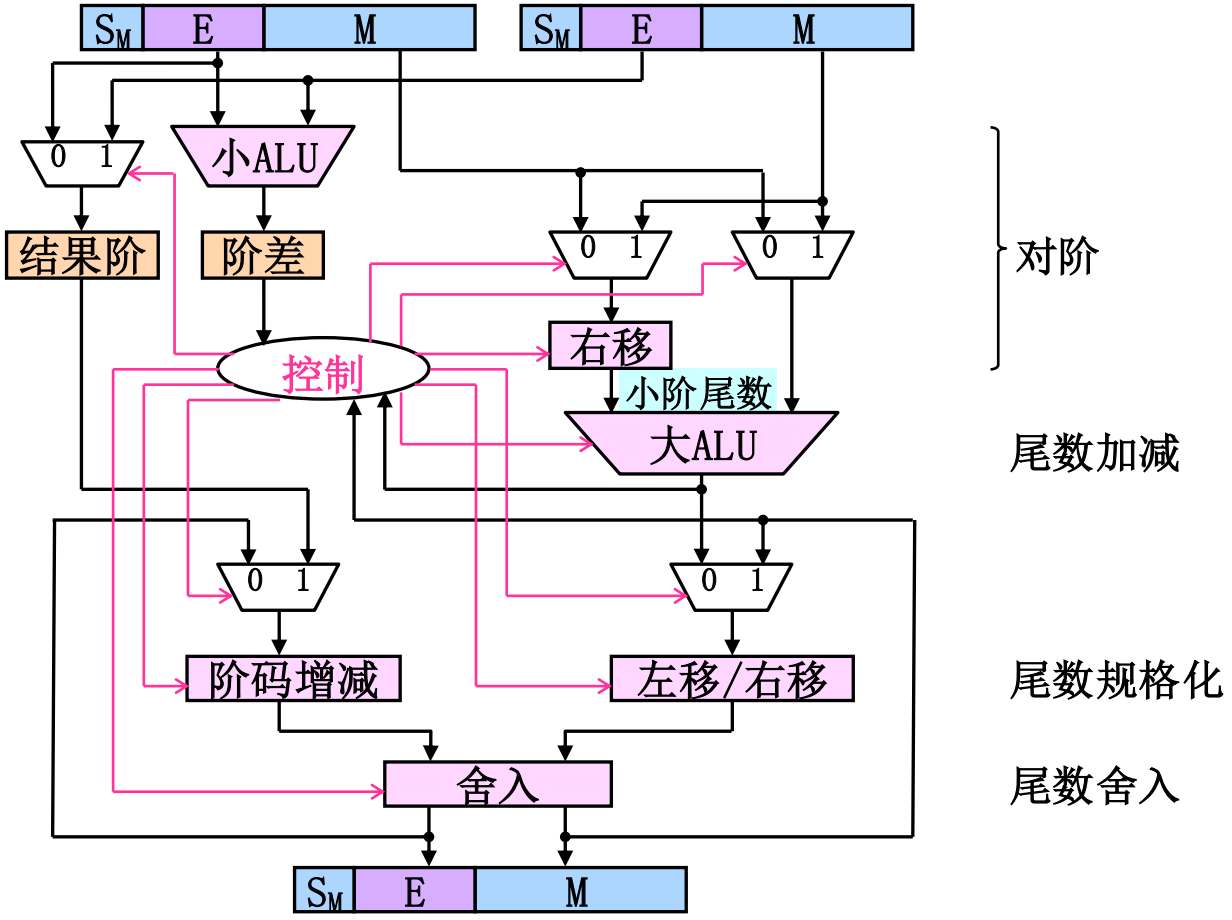
故$M^*_F = M''_F$(真值选择入)，M^*_F无需规格化，

$[M_F]_{补} = 1.0010$，$[E_F]_{补} = 010$

*逻辑实现：

思路—结构按运算步骤组织，尾数、阶分开处理，
尾数规格化、尾数舍入需判断溢出

结构—



2、浮点乘除运算　　（了解，不考）

***运算精度的保证：** 运算过程中，使用扩展浮点格式

└←最后才判断溢出←┘

如—IEEE 754标准的扩展格式为80位(1位数符＋15位阶码＋64位尾数)

***运算步骤：** 设$A=M_A\times 2^{E_A}$、$B=M_B\times 2^{E_B}$，$F=A\times B$或$A\div B=M_F\times 2^{E_F}$

①操作数预判—判断M_A及M_B是否符合要求(异常、操作数左规/右规)

└←同浮点加减运算

②阶码运算—$E_F=E_A+E_B$或E_A-E_B　←采用扩展格式，不会溢出

③尾数运算—$M_F=M_A\times M_B$或$M_A\div M_B$　←采用扩展格式，精度足够

④尾数规格化—使M_F为规格化数　←同浮点加减运算

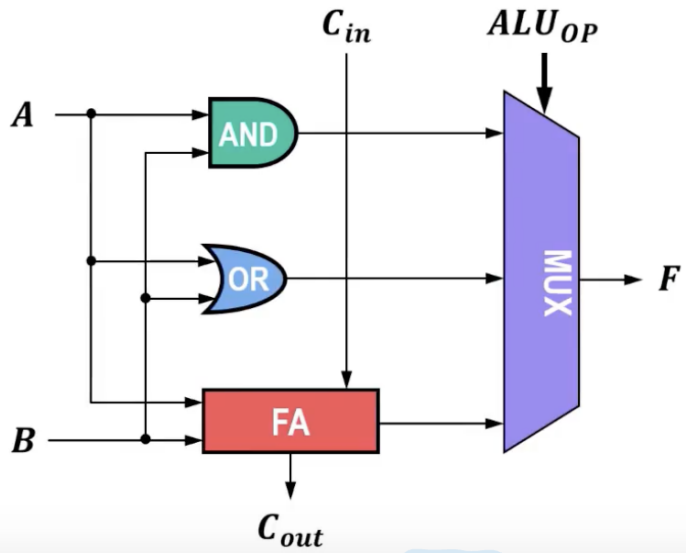
⑤尾数舍入—处理附加位　←同浮点加减运算

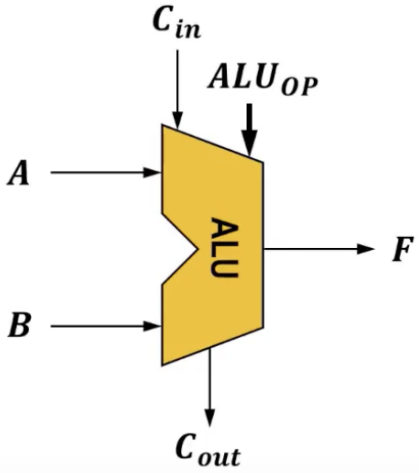
⑥溢出判断—将扩展格式转变为浮点格式，并判断溢出

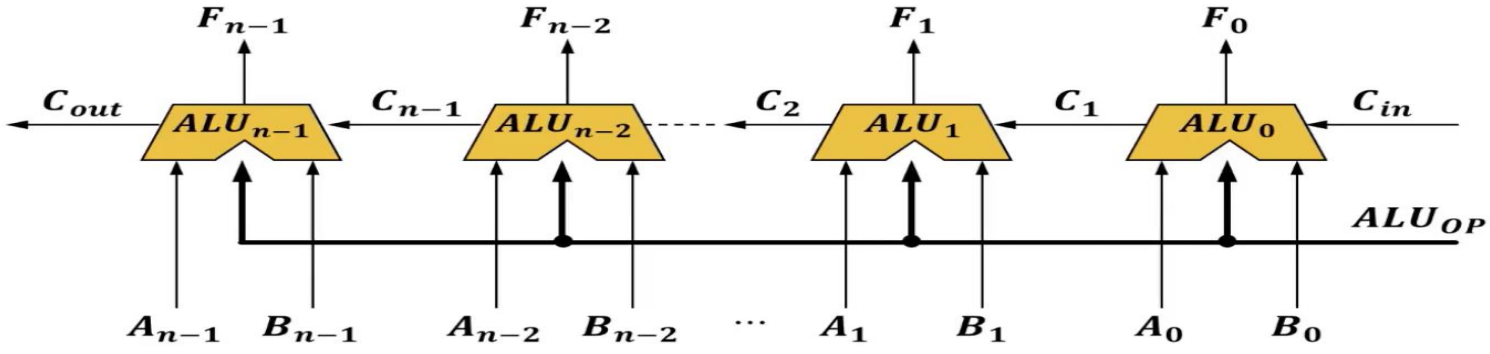
§2.5 运算器的组成

※主要内容：ALU的组成、运算器的组成(部件/互连)

【举例1】能够完成“与”、“或”、“加法”共3种运算的一位ALU。



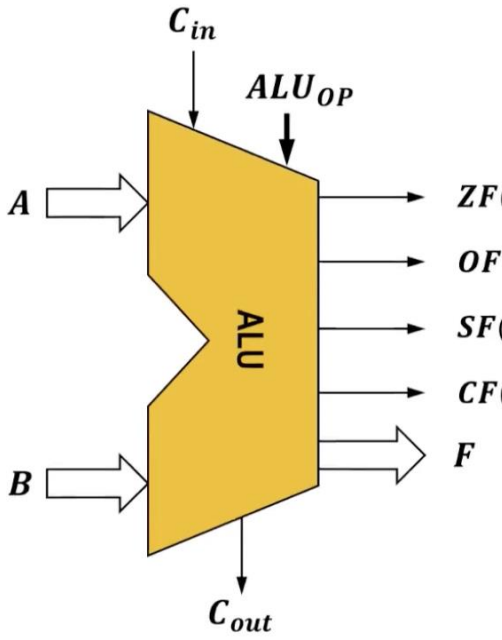




§2.5 运算器的组成

※主要内容：ALU的组成、运算器的组成(部件/互连)

常见状态标志位



$ZF(Zero\ Flag)$ ：为1表示运算结果为0

$OF(Overflow\ Flag)$ ：为1表示有符号运算溢出

$SF(Sign\ Flag)$ ：为1表示运算结果为负数

$CF(Carry\ Out\ Flag)$ ：表示加法进位输出、减法借位输出或逻辑左移操作的溢出位

§2.5 运算器的组成

※主要内容：ALU的组成、运算器的组成(部件/互连)

1、运算器的功能与组成

*运算器的功能： 实现ISA中所有指令的约定功能 ←组成是逻辑实现

指令功能示例— 功能包括数据操作及指令地址计算

指令名	指令功能
MOV	R[d]←M[R[s]]或M[R[s]]←R[d]，不保存标志， PC←PC+"1"
ADD	R[d]←R[d]＋R[s]，保存标志(运算产生)， PC←PC+"1"
XOR	R[d]←R[d]⊕M[R[s]]，保存标志(CF/OF＝0,其余运算产生)， PC←PC+"1"
SHL	R[d]←R[d]<<Imm，保存标志(运算产生)， PC←PC+"1"
JNZ	无目的操作数(不保存标志)，当ZF＝0时PC←R[s]，否则PC←PC+"1"
注：R[a]表示a#寄存器中内容，M[a]表示a#存储单元中内容，Imm为立即数	

指令功能特征— ①数据操作过程为取操作数+运算+写结果(含标志)

②操作数可存放在REG/MEM/IR(即指令字)中

③指令的计算功能常为一种 ←不同运算可复用器件

④结果标志应存放在REG中 ←用于后续指令

*运算器的组成：

包括运算部件&存放部件，功能满足所有条指令需求

运算部件— 为组合逻辑电路，有多种部件，每种部件含多个功能

（又称操作单元）　　└←基于能/否复用器件→┘

		数据表示	运算功能	运算方法	运算部件
数值数据	整数	定点(无符号)	加/减、移位，乘/除等	定点算术	ALU、移位器
		定点(有符号)			乘除法器等
	实数	浮点	加/减/乘/除等	浮点算术	FPU
非数值数据	逻辑数	(位)向量	与/或/非等	逻辑(按位)	ALU
	字符	定点(无符号)	>/≤/≠等	减法＋逻辑	ALU＋门电路
	其他	定点/向量	算逻/饱和等	算逻＋判断	MMX或SSE

存放部件— 为时序逻辑电路，有寄存器组REGs、状态寄存器PSR

（又称状态单元）　（存放数据/地址）　（存放标志等）

PSR—Program Status Register



2、运算器的部件组织

*设计步骤：汇总各指令的运算&操作需求、存放需求，

组织各部件功能(尽量复用器件)，设计各部件电路

思考：组合逻辑电路的设计步骤？ ①逻辑抽象(真值表)②逻辑化简③电路设计

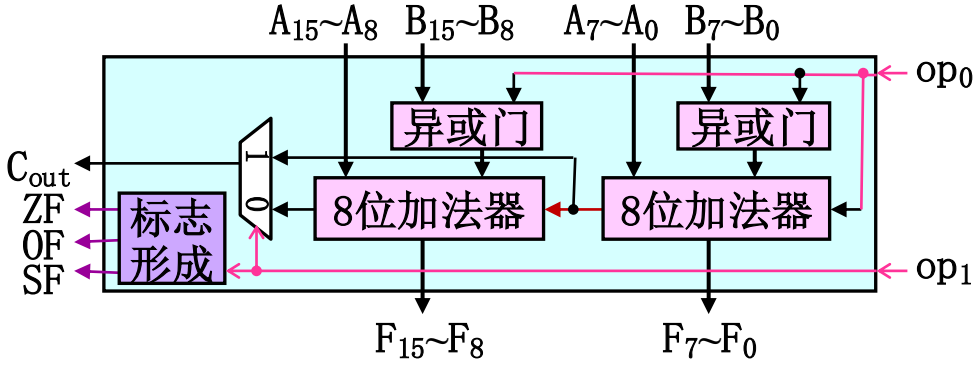
*设计示例：

例1：设计支持8位加减及16位加减的ALU，产生C_{out}及ZF/OF/SF

功能表—

op_1op_0	00	01	10	11
F_{15}~F_0	$A_{15\sim0}+B_{15\sim0}$	$A_{15\sim0}-B_{15\sim0}$	$F_{7\sim0}=A_{7\sim0}+B_{7\sim0}$	$F_{7\sim0}=A_{7\sim0}-B_{7\sim0}$
C_{out}	16位加法的C_{15}		8位加法的C_7	
ZF/OF/SF	按16位结果形成(稍后)		按8位结果形成(稍后)	

内部逻辑— 2个8位加减法器级联，2种C_{out}及标志的形成用op_1控制



标志的形成逻辑：

ZF=op_1? ~(F_7+…+F_0) :~(F_{15}+…+F_0)

OF=op_1?($C_7\oplus C_6$) : ($C_{15}\oplus C_{14}$)

SF=op_1? F_7:F_{15}

例2：①设计支持A&B、A⊕B的8位加法器；②基于①的结果，设计支持A±B、A±1、A&B、A⊕B的8位ALU，产生C_{out}及ZF/OF/SF

①加法器输出函数—

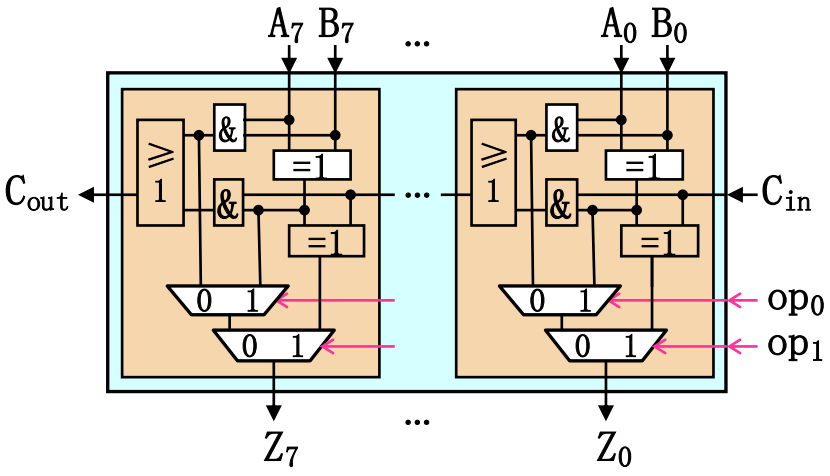
op_1op_0	00	01	1×
Z_7~Z_0	A&B	A⊕B	A+B

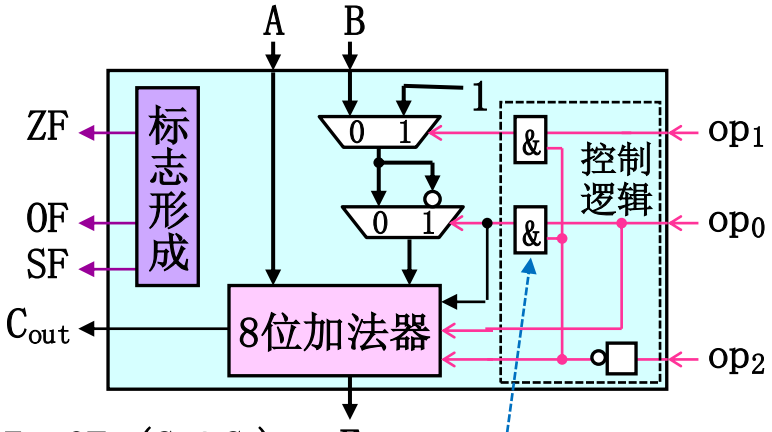
加法器内部逻辑—加法运算如下，逻辑运算复用加法元件，用op_1选择

②ALU输出函数—

op_2~op_0	000	001	010	011	100	101
F_{15}~F_0	A+B	A−B	A+1	A−1	A&B	A⊕B
C_{out}	加法器最高位进位					
ZF/OF/SF	按运算结果形成					

ALU内部逻辑—加减运算增加数据源(B或1)控制，逻辑运算复用加法器





C_{out}=C7，OF=(C_7⊕C_6)，
SF=F_7，ZF=~(F_7+…+F_0)

使加法器入端为B

3、运算器的部件互连

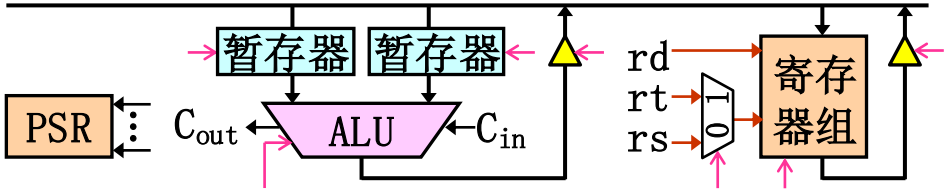
*数据引脚连接：有总线结构、专用结构2种

总线结构—各部件的入端/出端通过同一信号线连接

传输特性： 同时仅传送一个数据(仅一个部件可发送信号)

连接方法： 部件出端设置三态门，同一操作入/出端设置暂存器

仅一个信号直接连接总线→┘



思考①：ALU为何要暂存2个信号？

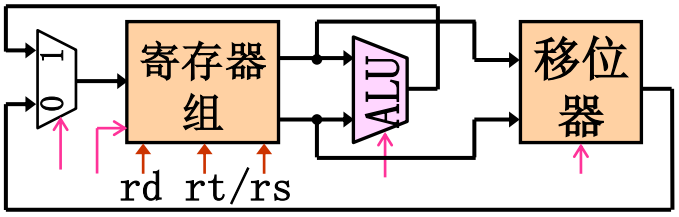
思考②：REG组为何不暂存信号？

专用结构—各部件的入端通过不同信号线连接其他部件的出端

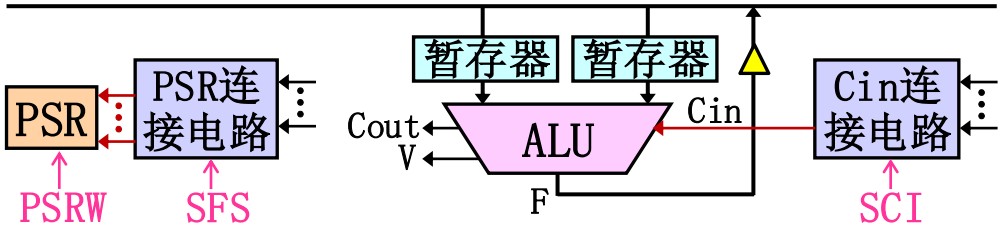
传输特性： 同时可传送多个数据

连接方法： 部件入端设置选择器，部件出端连接无限制

└←与指令功能相关 ←总线结构这无关



*其余引脚连接：

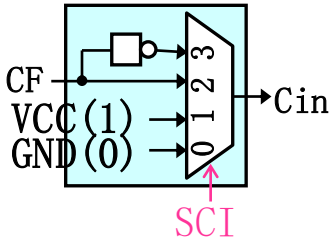


连接需求—汇总各指令功能

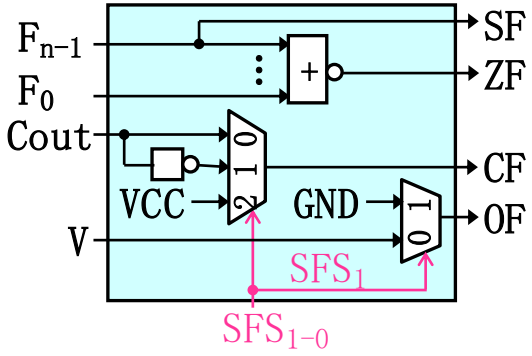
指令名	指令功能	标志的形成&保存	Cin	ZF	CF	OF	SF
ADD	R[d]←R[d]＋R[s]	运算产生	0	F_{n-1} +…+F_0	Cout	V	F_{n-1}
AUB	R[d]←R[d]－R[s]		1		Cout#		
ADC	R[d]←R[d]＋R[s]＋CF		CF		Cout		
SBB	R[d]←R[d]－R[s]－CF		CF#		Cout#		
XOR	R[d]←R[d]⊕R[s]	CF/OF=0，其余产生	*		0	0	
MOV	R[d]←M[Rs]	不保存	*	*	*	*	*

连接电路—分别实现连接需求的组合逻辑电路

SCI	Cin
00	0
01	1
10	CF
11	CF#

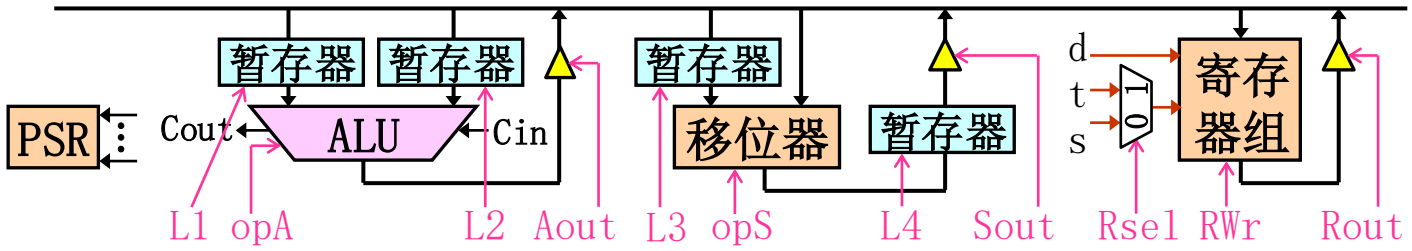


SFS	ZF	CF	OF	SF
00	F_{n-1} +…+F_0	Cout	V	F_{n-1}
01		Cout#	V	
10		0	0	
11	*	*	*	*



4、运算器的操作控制 （同时仅实现一条指令的功能）

*总线结构例：

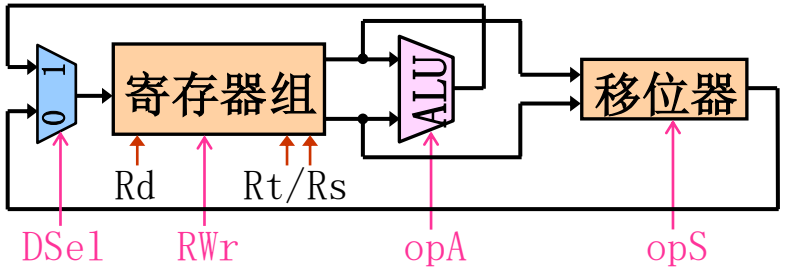


注：opA—00/01/10/11为+/-/&/⊕，opS—00/10/11为<</>>$_L$/>>$_A$

指令功能	实现步骤	L1	L2	opA	Aout	L3	opS	L4	Sout	Rsel	RegWr	Rout
R[d]←R[s]⊕R[t]	(1)取R[s]→L1	1	*	*	0	*	*	*	0	0	0	1
	(2)取R[t]→L2	0	1	*	0	*	*	*	0	1	0	1
	(3)⊕→R[d]	0	0	11	1	*	*	*	0	*	1	0

*专用结构例：

指令功能	DSel	RWr	opA	opS
R[d]←R[s]+R[t]	1	1	00	**
R[d]←R[s]⊕R[t]	1	1	11	**
R[d]←R[s]>>$_L$R[t]	0	1	**	10



注：opA及opS功能同上

*影响指令实现性能的因素：

部件的组织，部件互连的结构

（～操作延迟） （～实现步数）

作业2-4：P87—37、40